

UNIT-1

Introduction to Database Management System

- As the name suggests, the database management system consists of two parts. They are:
 1. Database and
 2. Management System

What is Database?

- To find out what database is, we have to start from data, which is the basic building block of any DBMS
- A database is an organized collection of structured information, or data, typically stored electronically in a computer system.
- A database is usually controlled by a database management system (DBMS). Together, the data and the DBMS, along with the applications that are associated with them, are referred to as a database system

What is Data & Information?

- **Data**
 - Facts, figures, statistics etc. having no particular meaning (e.g. 101, ABC, 82 etc).
 - Data is not significant to business
 - Data are atomic level pieces of information. It can be in the form of numbers, characters, symbols, or even pictures.
 - Difficult to understand properly
 - Data May not be in order
- **Information**
 - Information is a processed form of data. For example: AGE 32
 - Information is significant to business
 - Information helps in decision making
 - Easy to understand
 - Information should be in the order

What is Record?

- **Collection of related data items**, e.g. in the above example the three data items had no meaning. But if we organize them in the following way, then they collectively represent meaningful information
-

EID	ENAME	AGE
101	ABC	82

Table or Relation: Collection of related records.

EID	ENAME	REDATE	SALARY	DEPTNO
101	ABC	16/02/2023	34000	10
102	DEF	15/02/2022	38000	20
103	FGH	12/06/2020	19000	30

- The columns of this relation are called **Fields**, **Attributes** or **Domains**. The rows are called **Tuples** or **Records**.

Database: Collection of related relations. Consider the following collection of tables

TABLE-1					TABLE-2		
EID	ENAME	HIREDATE	SALARY	DEPTNO	DEPTNO	DNAME	LOCATION_ID
1001	ABC	12/02/2000	18000	10	10	HR	2001
1002	XYZ	13/06/2022	25000	20	20	FINANCE	2002
1003	FGH	11/05/2019	18500	30	30	ACCOUNTS	2001
1004	KLM	10/01/2021	14800	10			
TABLE-3					TABLE-4		
LID	STREET_ADD	POSTAL_CD	CITY	CID	CID	COUNTRY_NAME	REGION_ID
2001	12-98 Victoria street	2901	Sydney	AU	AU	Australia	3
2002	2007,Zagora st	50090	South Brunswick	US	US	United States of America	2
2003	2017,Shinjuku-ku	1689	Tokyo	JP	JP	Japan	3
2004	6092,Boxwood St	80925	Whitehorse	CA	CA	Canada	2

- We now have a collection of 4 tables. They can be called a “related collection” because we can clearly find out that there are some common attributes existing in a selected pair of tables. Because of these common attributes we may combine the data of two or more tables together to find out the complete details of an employee.

A database in a DBMS could be viewed by lots of different people with different responsibilities.

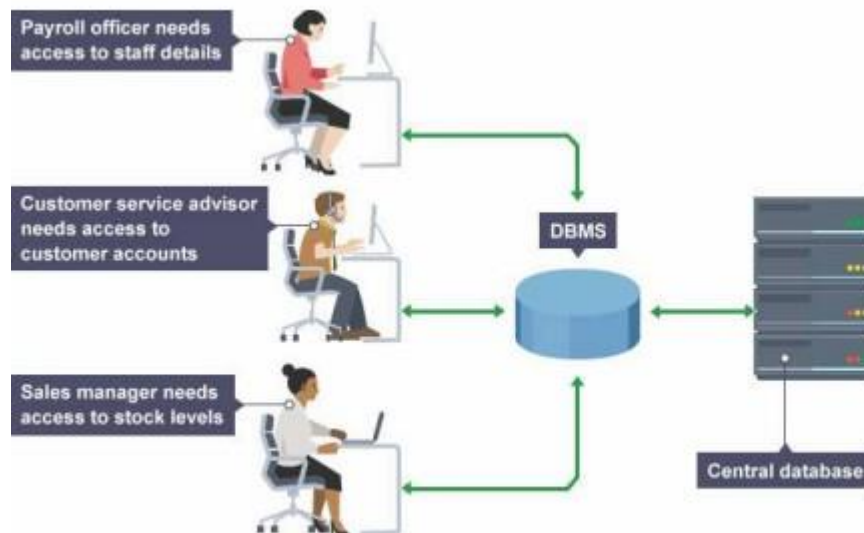


Figure 1.1: Employees are accessing Data through DBMS

- For example, within a company there are different departments, as well as customers, who each need to see different kinds of data. Each employee in the company will have different levels of access to the database with their own customized front-end application.
- In a database, data is organized strictly in row and column format. The rows are called Tuple or Record. The data items within one row may belong to different data types. On the other hand, the columns are often called Domain or Attribute. All the data items within a single attribute are of the same data type.

What is Management System?

- A database-management system (DBMS) is a **collection of interrelated data** and a **set of programs** to access those data. This is a collection of related data with an implicit meaning and hence is a database.
- The primary goal of a DBMS is to provide a way to store and retrieve database information that is both convenient and efficient. By data, we mean known facts that can be recorded and that have implicit meaning

- The management system is important because without the existence of some kind of rules and regulations it is not possible to maintain the database.
- We have to select the particular attributes which should be included in a particular table; the common attributes to create relationship between two tables; if a new record has to be inserted or deleted then which tables should have to be handled etc.
- These issues must be resolved by having some kind of rules to follow in order to maintain the integrity of the database.
- Database systems are designed to manage large bodies of information. **Management of data involves both defining structures for storage of information and providing mechanisms for the manipulation of information.**
- In addition, the database system must ensure the **safety of the information** stored, despite system crashes or attempts at unauthorized access. If data are to be shared among several users, the system must avoid possible anomalous results.
- Because information is so important in most organizations, computer scientists have developed a large body of concepts and techniques for managing data.

Database Management System (DBMS) and Its Applications:

- **A Database management system is a computerized record-keeping system.** It is a **repository** or a **container** for collection of computerized data files. The overall purpose of DBMS is to allow users to **define, store, retrieve and update** the information contained in the database **on demand**.
- Information can be anything that is of significance to an individual or organization.
- Databases touch all aspects of our lives. **Some of the major areas of application are as follows:**
 1. **Railway Reservations System**
 2. **Library Management System**
 3. **Banking**
 4. **Educational Institutions**
 5. **Social Media Websites**

The different applications of DBMS are as follows –

Railway Reservation System – It is used to keep record of booking of tickets, departure of the train and the status of arrival and give updates to the passengers with the help of a database.

Library Management System – There will be so many numbers of books in the library and it is very hard to keep a record of all the books in a register or a copy. So, DBMS is necessary to keep track of all the book records, issue dates, name of the books, author and maintain the records.

Banking – we are doing a lot of transactions daily without directly going to the banks. The only reason is the usage of databases and it manages all the data of the customers over the database.

Educational Institutions – All the examinations and the data related to the students maintained over the internet with the help of a database management system. It contains registration details of the student, results, grades and courses available. All these works can be done online without visiting an institution.

Social Media Websites – By filling the required details we are able to access social media platforms. Many users daily sign up for social websites such as Facebook, Pinterest and Instagram. All the information related to the users are stored and maintained with the help of DBMS.

DBMS Vendors and their Products

vendor	product
IBM	–DB2/MVS –DB2/UDB –DB2/400 –Informix Dynamic Server (IDS)
Microsoft	–Access –SQL Server –DesktopEdition(MSDE)
Open Source	–MySQL –PostgreSQL
Oracle	–Oracle DBMS –RDB
Sybase	–Adaptive Server Enterprise (ASE) –Adaptive Server Anywhere (ASA) –Watcom

Purpose of Database Systems

- Database systems arose in response to early methods of computerized management of commercial data. As an example of such methods, typical of the 1960s, consider part of a university organization that, among other data, keeps information about all instructors, students, departments, and course offerings.
- One way to keep the information on a computer is to store it in operating system files. To allow users to manipulate the information, the system has a number of application programs that manipulate the files, including programs to:
 - Add new students, instructors, and courses
 - Register students for courses and generate class rosters
 - Assign grades to students, compute grade point averages (GPA), and generate transcripts
- System programmers wrote these application programs to meet the needs of the university.
- New application programs are added to the system as the need arises. For example, suppose that a university decides to create a new major (say, computer science).As a result, the university creates a new department and creates new permanent files (or adds information to existing files) to record information about all the instructors in the department, students in that major, course offerings, degree requirements, etc.
- The university may have to write new application programs to deal with rules specific to the new major. New application programs may also have to be written to handle new rules in the university. Thus, as time goes by, the system acquires more files and more application programs
- This typical file-processing system is supported by a conventional operating system. The system stores permanent records in various files, and it needs different application programs to extract records from, and add records to, the appropriate files. Before database management systems (DBMSs) were introduced, organizations usually stored information in such systems. Keeping organizational information in a file processing system has a number of major disadvantages:
 - **Data redundancy and inconsistency.**
 - Since different programmers create the files and application programs over a long period, the various files are likely to have different structures and the programs may be written in several

programming languages. Moreover, the same information may be duplicated in several places (files).

- For example, if a student has a double major (say, music and mathematics) the address and telephone number of that student may appear in a file that consists of student records of students in the Music department and in a file that consists of student records of students in the Mathematics department.

This redundancy leads to higher storage and access cost. In addition, it may lead to data inconsistency; that is, the various copies of the same data may no longer agree. For example, a changed student address may be reflected in the Music department records but not elsewhere in the system.

Difficulty in accessing data.

Suppose that one of the university clerks needs to find out the names of all students who live within a particular postal-code area. The clerk asks the data-processing department to generate such a list. Because the designers of the original system did not anticipate this request, there is no application program on hand to meet it.

There is, however, an application program to generate the list of all students. The university clerk has now two choices: either obtain the list of all students and extract the needed information manually or ask a programmer to write the necessary application program. Both alternatives are obviously unsatisfactory.

Suppose that such a program is written, and that, several days later, the same clerk needs to trim that list to include only those students who have taken at least 60 credit hours. As expected, a program to generate such a list does not exist. Again, the clerk has the preceding two options, neither of which is satisfactory. The point here is that conventional file-processing environments do not allow needed data to be retrieved in a convenient and efficient manner. More responsive data-retrieval systems are required for general use

Data isolation.

Because data are scattered in various files, and files may be in different formats, writing new application programs to retrieve the appropriate data is difficult.

Integrity problems.

The data values stored in the database must satisfy certain types of consistency

Constraints.

Suppose the university maintains an account for each department, and records the balance amount in each account. Suppose also that the university requires that the account balance of a department may never fall below zero. Developers enforce these constraints in the system by adding appropriate code in the various application programs. However, when new constraints are added, it is difficult to change the programs to enforce them. The problem is compounded when constraints involve several data items from different files

Atomicity problems.

A computer system, like any other device, is subject to failure. In many applications, it is crucial that, if a failure occurs, the data be restored to the consistent state that existed prior to the failure.

Consider a program to transfer \$500 from the account balance of department A to the account balance of department B. If a system failure occurs during the execution of the program, it is possible that the \$500 was removed from the balance of department A but was not credited to the balance of department B, resulting in an inconsistent database state. Clearly, it is essential to database consistency that either both the credit and debit occur, or that neither occur.

That is, the funds transfer must be atomic—it must happen in its entirety or not at all. It is difficult to ensure atomicity in a conventional file-processing system.

Concurrent-access anomalies.

For the sake of overall performance of the system and faster response, many systems allow multiple users to update the data simultaneously. Indeed, today, the largest Internet retailers may have millions of accesses per day to their data by shoppers. In such an environment, interaction of concurrent updates is possible and may result in inconsistent data.

Consider department A, with an account balance of \$10,000. If two department clerks debit the account balance (by say \$500 and \$100, respectively) of department A at almost exactly the same time, the result of the concurrent executions may leave the budget in an incorrect (or inconsistent) state. Suppose that the programs executing on behalf of each withdrawal read the old balance, reduce that value by the amount being withdrawn, and write the result back.

If the two programs run concurrently, they may both read the value \$10,000, and write back \$9500 and \$9900, respectively. Depending on which one writes the value last, the account balance of department A may contain either \$9500 or \$9900, rather than the correct value of \$9400. To guard against this possibility, the system must maintain some form of supervision. But supervision is difficult to provide because data may be accessed by many different application programs that have not been coordinated previously.

As another example, suppose a registration program maintains a count of students registered for a course, in order to enforce limits on the number of students registered. When a student registers, the program reads the current count for the courses, verifies that the count is not already at the limit, adds one to the count, and stores the count back in the database. Suppose two students register concurrently, with the count at (say) 39. The two program executions may both read the value 39, and both would then write back 40, leading to an incorrect increase of only 1, even though two students successfully registered for the course and the count should be 41. Furthermore, suppose the course registration limit was 40; in the above case both students would be able to register, leading to a violation of the limit of 40 students

Security problems.

Not every user of the database system should be able to access all the data. For example, in a university, payroll personnel need to see only that part of the database that has financial information. They do not need access to information about academic records. But, since application programs are added to the file-processing system in an ad hoc manner, enforcing such security constraints is difficult. These difficulties, among others, prompted the development of database systems. In what follows, we shall see the concepts and algorithms that enable database systems to solve the problems with file-processing systems.

Advantages of DBMS:

- **Controlling of Redundancy:**
 - Data redundancy refers to the duplication of data (i.e storing same data multiple times). In a database system, by having a centralized database and centralized control of data by the DBA the unnecessary duplication of data is avoided. It also eliminates the extra time for processing the large volume of data. It results in saving the storage space.
- **Improved Data Sharing :**
 - DBMS allows a user to share the data in any number of application programs.
- **Data Integrity:**
 - Integrity means that the data in the database is accurate. Centralized control of the data helps in permitting the administrator to define integrity constraints to the data in the database. For example: in customer database we can enforce integrity that it must accept the customer only from Nandyal and kurnool city.
- **Security :**
 - Having complete authority over the operational data, enables the DBA in ensuring that the only mean of access to the database is through proper channels. The DBA can define authorization checks to be carried out whenever access to sensitive data is attempted.
- **Data Consistency:**
 - By eliminating data redundancy, we greatly reduce the opportunities for inconsistency. For example: is a customer address is stored only once, we cannot have disagreement on the stored values. Also updating data values is greatly simplified when each value is stored in one place only. Finally, we avoid the wasted storage that results from redundant data storage.
- **Efficient Data Access :**
 - In a database system, the data is managed by the DBMS and all access to the data is through the DBMS providing a key to effective data processing
- **Enforcements of Standards :**
 - With the centralized of data, DBA can establish and enforce the data standards which may include the naming conventions, data quality standards etc.
- **Data Independence :**
 - In a database system, the database management system provides the interface between the application programs and the data. When changes are made to the data representation, the meta data obtained by the DBMS is changed but the DBMS is continues to provide the data to application program in the previously used way.
 - The DBMS handles the task of transformation of data wherever necessary. Reduced Application Development and Maintenance Time: DBMS supports many important functions that are common to many applications, accessing data stored in the DBMS, which facilitates the quick development of application.

Disadvantages of DBMS

- **It is bit complex.**

- a. Since it supports multiple functionality to give the user the best, the underlying software has become complex. The designers and developers should have thorough knowledge about the software to get the most out of it.
- b. Because of its complexity and functionality, it uses large amount of memory. It also needs large memory to run efficiently.
- c. DBMS system works on the centralized system, i.e.; all the users from all over the world access this database. Hence any failure of the DBMS, will impact all the users.
- d. DBMS is generalized software, i.e.; it is written work on the entire systems rather specific one. Hence some of the application will run slow.

Data Dictionary

- We can define a data dictionary as a DBMS component that stores the definition of **data characteristics** and relationships. You may recall that such “**data about data**” were labeled metadata.
- The DBMS data dictionary provides the DBMS with its self describing characteristic. In effect, the data dictionary resembles /, and is a crucial element in the data administration function.
- The two main types of data dictionary exist, **integrated and stand alone**.
- An integrated data dictionary is included with the DBMS. For example, all relational DBMSs include a built in data dictionary or system catalog that is frequently accessed and updated by the RDBMS.
- Other DBMSs especially older types, do not have a built in data dictionary instead the DBA may use third party stand alone data dictionary systems.
- **Data dictionaries can also be classified as active or passive.**
 - An active data dictionary is automatically updated by the DBMS with every database access, thereby keeping its access information **up-to-date**.
 - A passive data dictionary is not updated automatically and usually requires a **batch process to be run**. Data dictionary access information is normally used by the DBMS for query optimization purpose.
- The data dictionary's main function is to store the description of all objects that interact with the database. Integrated data dictionaries tend to limit their metadata to the data managed by the DBMS.
- Stand alone data dictionary systems are more usually more flexible and allow the DBA to describe and manage all the organization's data, whether or not they are computerized. Whatever the data dictionary's format, its existence provides database designers and end users with a much improved ability to communicate. In addition, the data dictionary is the tool that helps the DBA to resolve data conflicts.
- Although, there is no standard format for the information stored in the data dictionary several features are common.
- For example, the data dictionary typically stores descriptions of all: Data elements that are define in all tables of all databases. Specifically the data dictionary stores the name, data types, display formats, internal storage formats, and validation rules. The data dictionary tells
- Where an element is used, by whom it is used and so on. Tables define in all databases. For example, the data dictionary is likely to store the name of the table
- Creator, the date of creation access authorizations, the number of columns, and so on.
 - Indexes define for each database tables. For each index the DBMS stores at least the index name the attributes used, the location, specific index characteristics and the creation date.
 - Define databases: who created each database, the date of creation where the database is located, who the DBA is and so on.
 - End users and The Administrators of the data base
 - Programs that access the database including screen formats, report formats application formats, SQL queries and so on.
 - Access authorization for all users of all databases.
 - Relationships among data elements which elements are involved: whether the relationship are mandatory or optional, the connectivity and cardinality and so on.

Some Examples:

Ex1:

CONNECT system/your_password

```
SELECT owner, table_name, tablespace_name
FROM dba_tables
WHERE table_name='BOOKS';
```

Ex2:

```
SELECT table_name FROM user_tables;
```

Ex3:

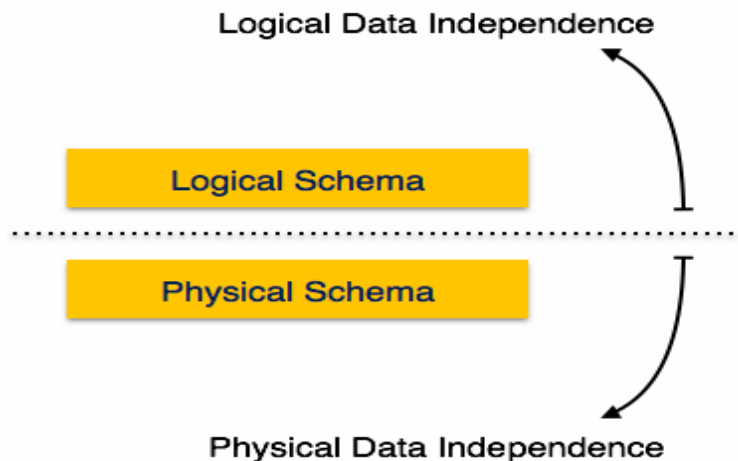
```
SELECT
  table_name
FROM
  all_tables
WHERE
  table_name LIKE 'EMP%';
```

Ex4:

```
SELECT
  table_name
FROM
  dba_tables
WHERE
  table_name LIKE 'EMP%'
AND
  owner like 'ROBERT%';
```

Data Independence

- A database system normally contains a lot of data in addition to users' data. For example, it stores data about data, known as metadata, to locate and retrieve data easily. It is rather difficult to modify or update a set of metadata once it is stored in the database. But as a DBMS expands, it needs to change over time to satisfy the requirements of the users. If the entire data is dependent, it would become a tedious and highly complex job.



- **Metadata itself follows a layered architecture**, so that **when we change data at one layer, it does not affect the data at another level**. This **data is independent but mapped to each other**.

Logical Data Independence

- Logical data is **data about database**, that is, it stores information about how data is managed inside. For example, a table (relation) stored in the database and all its constraints, applied on that relation.
- Logical data independence is a kind of mechanism, which liberalizes itself from actual data stored on the disk. If we do some changes on table format, it should not change the data residing on the disk.

Physical Data Independence

- All the schemas are logical, and the actual data is stored in bit format on the disk. Physical data independence is the power to change the physical data without impacting the schema or logical data.
- For example, in case we want to change or upgrade the storage system itself – suppose we want to replace hard-disks with SSD – it should not have any impact on the logical data or schemas.

View of Data

- A database system is a collection of interrelated data and a set of programs that allow users to access and modify these data. A major purpose of a database system is to provide users with an **abstract view of the data**. That is, the **system hides certain details of how the data are stored and maintained**.

Data Abstraction

- For the system to be usable, it must retrieve data efficiently. The need for efficiency has led designers to use complex data structures to represent data in the database. Since many database-system users are not computer trained, developers hide the complexity from users through several levels of abstraction, to simplify users' interactions with the system:

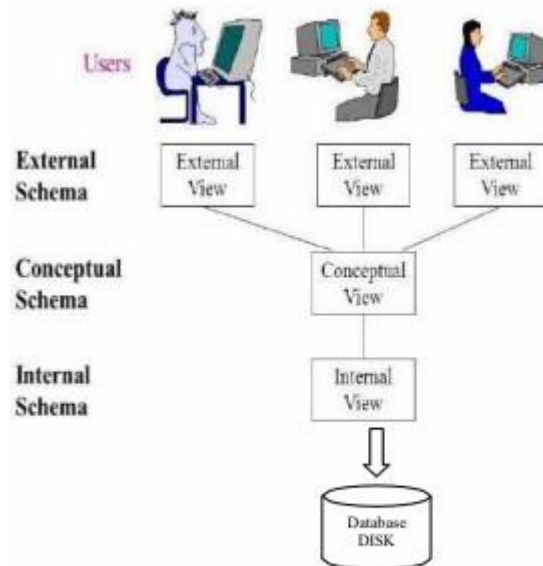


Figure 1.2 : Levels of Abstraction in a DBMS

Physical Data Independence:

- **Physical Data Independence** means changing the physical level without affecting the logical level or conceptual level. Using this property, we can change the storage device of the database without affecting the logical schema.

Physical level (or Internal View / Schema):

- The lowest level of abstraction describes how the data are actually stored. The physical level describes complex low-level data structures in detail.

Logical level (or Conceptual View / Schema):

- The next-higher level of abstraction describes what data are stored in the database, and what relationships exist among those data. The logical level thus describes the entire database in terms of a small number of relatively simple structures. Although implementation of the simple structures at the logical level may involve complex physical-level structures, the user of the logical level does not need to be aware of this complexity. This is referred to as physical data independence. Database administrators, who must decide what information to keep in the database, use the logical level of abstraction

View level (or External View / Schema):

- The highest level of abstraction describes only part of the entire database. Even though the logical level uses simpler structures, complexity remains because of the variety of information stored in a large database. Many users of the database system do not need all this information; instead, they need to access only a part of the database. The view level of abstraction exists to simplify their interaction with the system. **The system may provide many views for the same database.** Figure 1.2 shows the relationship among the three levels of abstraction. An analogy to the concept of data types in programming languages may clarify the distinction among levels of abstraction. Many high-level programming languages support the notion of a structured type. For example, we may describe a record as follows:

```
type instructor = record
    ID : char (5);
    name : char (20);
    dept name : char (20);
```

```
salary : numeric (8,2);  
end;
```

- This code defines a new record type called instructor with four fields. Each field has a name and a type associated with it. A university organization may have several such record types, including
 - department, with fields dept_name, building, and budget
 - course, with fields course_id, title, dept_name, and credits
 - student, with fields ID, name, dept_name, and tot_cred
- **At the physical level**, an instructor, department, or student record can be described as a block of consecutive storage locations. The compiler hides this level of detail from programmers. Similarly, the database system hides many of the lowest-level storage details from database programmers. Database administrators, on the other hand, may be aware of certain details of the physical organization of the data.
- **At the logical level**, each such record is described by a type definition, as in the previous code segment, and the interrelationship of these record types is defined as well. Programmers using a programming language work at this level of abstraction. Similarly, database administrators usually work at this level of abstraction.
- **Finally, at the view level**, computer users see a set of application programs that hide details of the data types. At the view level, several views of the database are defined, and a database user sees some or all of these views. In addition to hiding details of the logical level of the database, the views also provide a security mechanism to prevent users from accessing certain parts of the database. For example, clerks in the university registrar office can see only that part of the database that has information about students; they cannot access information about salaries of instructors.

Instances and Schemas

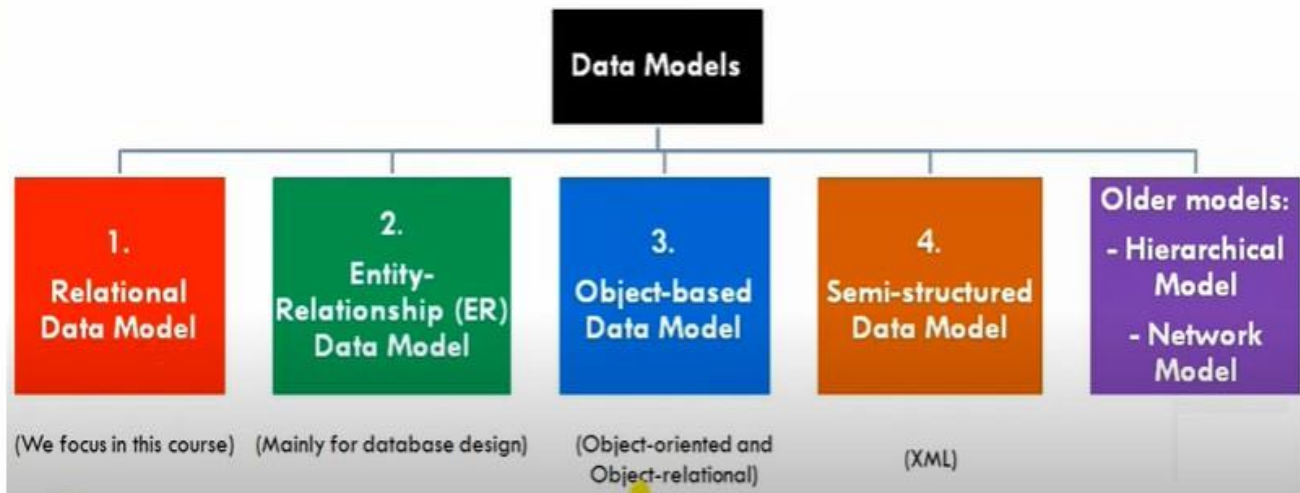
- **Databases change over time as information is inserted and deleted. The collection of information stored in the database at a particular moment is called an instance of the database.**
- **The overall design of the database is called the database schema. Schemas are changed infrequently**, if at all. The concept of database schemas and instances can be understood by analogy to a program written in a programming language.
- A database schema corresponds to the variable declarations (along with associated type definitions) in a program. Each variable has a particular value at a given instant. The values of the variables in a program at a point in time correspond to an instance of a database schema.
- Database systems have several schemas, partitioned according to the levels of abstraction. The physical schema describes the database design at the physical level, while the logical schema describes the database design at the logical level. A database may also have several schemas at the view level, sometimes called subschemas, which describe different views of the database. Of these, the logical schema is by far the most important, in terms of its effect on application programs, since programmers construct applications by using the logical schema. The physical schema is hidden beneath the logical schema, and can usually be changed easily without affecting application programs. Application programs are said to exhibit physical data independence if they do not depend on the physical schema, and thus need not be rewritten if the physical schema changes

Data Models

- Underlying **the structure of a database is the data model**: a collection of conceptual tools for **describing data, data relationships, data semantics, and consistency constraints**. A data model provides a way to describe the design of a database at the **physical, logical, and view levels**.

The data models can be classified into four different categories

Types of Data Models



Entity-Relationship Model

- The entity-relationship (E-R) data model uses a collection of basic objects, called entities, and relationships among these objects.
- An entity is a “thing” or “object” in the real world that is distinguishable from other objects. The entity relationship model is widely used in database design.

Relational Model

- The relational model uses a collection of tables to represent both data and the relationships among those data. Each table has multiple columns, and each column has a unique name. Tables are also known as relations. The relational model is an example of a **record-based model**. Record-based models are so named because the database is structured in fixed-format records of several types. Each table contains records of a particular type. Each record type defines a fixed number of fields, or attributes. The columns of the table correspond to the attributes of the record type. **The relational data model is the most widely used data model**, and a vast majority of current database systems are based on the relational model.

Object-Based Data Model

- Object-oriented programming (especially in Java, C++, or C#) has become the dominant software-development methodology. This led to the development of an object-oriented data model that can be seen as extending the E-R model with notions of encapsulation, methods (functions), and object identity. The object-relational data model combines features of the object-oriented data model and relational data model.

Semi-structured Data Model

- The semi-structured data model permits the specification of data where individual data items of the same type may have different sets of attributes. This is in contrast to the data models mentioned earlier, where every data item of a particular type must have the same set of attributes. The Extensible Markup Language (XML) is widely used to represent semi-structured data. Historically, the network data model and the hierarchical data model preceded the relational data model. These models were tied closely to the underlying implementation, and complicated the task of modeling data. As a result they are used little now, except in old database code that is still in service in some places.

Database Languages

- A database system provides a **data-definition language to specify the database schema** and a **data-manipulation language to express database queries and updates**. In practice, the data definition and data-manipulation languages are not two separate languages; instead they simply form parts of a single database language, such as the widely used **SQL language**.

Data-Manipulation Language

- A data-manipulation language (DML) is a language that enables users to access or manipulate data as organized by the appropriate data model. The types of access are:
 - Retrieval of information stored in the database
 - Insertion of new information into the database
 - Deletion of information from the database
 - Modification of information stored in the database
- There are basically two types:
 - **Procedural DMLs**
 - Require a user to specify what data are needed and how to get those data.
 - **Declarative DMLs (also referred to as nonprocedural DMLs)**
 - Require a user to specify what data are needed without specifying how to get those data. Declarative DMLs are usually easier to learn and use than are procedural DMLs. However, since a user does not have to specify how to get the data, the database system has to figure out an efficient means of accessing data. A query is a statement requesting the retrieval of information. The portion of a DML that involves information retrieval is called a query language. Although technically incorrect, it is common practice to use the terms query language and data-manipulation language synonymously

Data-Definition Language (DDL)

- We specify a database schema by a set of definitions expressed by a special language called a data-definition language (DDL). The DDL is also used to specify additional properties of the data. We specify the storage structure and access methods used by the database system by a set of statements in a special type of DDL called a data storage and definition language. These statements define the implementation details of the database schemas, which are usually hidden from the users. The data values stored in the database must satisfy certain consistency constraints.
- For example, suppose the university requires that the account balance of a department must never be negative. The DDL provides facilities to specify such constraints. The database system checks these constraints every time the database is updated. In general, a constraint can be an arbitrary predicate pertaining to the database. However, arbitrary predicates may be costly to test. Thus, database systems implement integrity constraints that can be tested with minimal overhead

Domain Constraints

- A domain of possible values must be associated with every attribute (for example, integer types, character types, date/time types). Declaring an attribute to be of a particular domain acts as a constraint on the values that it can take. Domain constraints are the most elementary form of integrity constraint. They are tested easily by the system whenever a new data item is entered into the database.

Referential Integrity

- There are cases where we wish to ensure that a value that appears in one relation for a given set of attributes also appears in a certain set of attributes in another relation (referential integrity).
- For example, the department listed for each course must be one that actually exists. More precisely, the dept name value in a course record must appear in the dept name attribute of some record of the department relation. Database modifications can cause violations of referential integrity. When a referential-integrity constraint is violated, the normal procedure is to reject the action that caused the violation

Assertions

- An assertion is any condition that the database must always satisfy. Domain constraints and referential-integrity constraints are special forms of assertions. However, there are many constraints that we cannot express by using only these special forms. For example, "Every department must have at least five courses offered every semester" must be expressed as an assertion. When an assertion is created, the system tests it for validity. If the assertion is valid, then any future modification to the database is allowed only if it does not cause that assertion to be violated

Authorization

- We may want to differentiate among the users as far as the type of access they are permitted on various data values in the database. These differentiations are expressed in terms of authorization, the most common being: read authorization, which allows reading, but not modification, of data; insert authorization, which allows insertion of

new data, but not modification of existing data; update authorization, which allows modification, but not deletion, of data; and delete authorization, which allows deletion of data. We may assign the user all, none, or a combination of these types of authorization.

- The DDL, just like any other programming language, gets as input some instructions (statements) and generates some output. The output of the DDL is placed in the data dictionary, which contains metadata—that is, data about data. The data dictionary is considered to be a special type of table that can only be accessed and updated by the database system itself (not a regular user). The database system consults the data dictionary before reading or modifying actual data.

Database Administrators and Database Users

- A primary goal of a database system is to retrieve information from and store new information in the database.
- People who work with a database can be categorized as database users or database administrators.

Database Users and User Interfaces

- Database users are categorized based up on their interaction with the database. These are seven types of database users in DBMS.

Database Administrator (DBA) :

- Database Administrator (DBA) is a person/team who defines the schema and also controls the 3 levels of database. The DBA will then create a new account id and password for the user if he/she need to access the database. DBA is also responsible for providing security to the database and he allows only the authorized users to access/modify the data base. DBA is responsible for the problems such as security breaches and poor system response time.
 - DBA also monitors the recovery and backup and provide technical support.
 - The DBA has a DBA account in the DBMS which called a system or superuser account.
 - DBA repairs damage caused due to hardware and/or software failures.
 - DBA is the one having privileges to perform DCL (Data Control Language) operations such as GRANT and REVOKE, to allow/restrict a particular user from accessing the database.

Naive / Parametric End Users :

- Parametric End Users are the unsophisticated who don't have any DBMS knowledge but they frequently use the database applications in their daily life to get the desired results. For examples, Railway's ticket booking users are naive users. Clerks in any bank are a naive user because they don't have any DBMS knowledge but they still use the database and perform their given task.

System Analyst:

- System Analyst is a user who analyzes the requirements of parametric end users. They check whether all the requirements of end users are satisfied.

Sophisticated Users:

- Sophisticated users can be engineers, scientists, business analyst, who are familiar with the database. They can develop their own database applications according to their requirement. They don't write the program code but they interact the database by writing SQL queries directly through the query processor.

Database Designers:

- Data Base Designers are the users who design the structure of database which includes tables, indexes, views, triggers, stored procedures and constraints which are usually enforced before the database is created or populated with data. He/she controls what data must be stored and how the data items to be related. It is responsibility of Database Designers to understand the requirements of different user groups and then create a design which satisfies the need of all the user groups.

Application Programmers:

- Application Programmers also referred as System Analysts or simply Software Engineers, are the back-end programmers who writes the code for the application programs. They are the computer professionals. These programs could be written in Programming languages such as Visual Basic, Developer, C, FORTRAN, COBOL etc. Application programmers design, debug, test, and maintain set of programs called "canned transactions" for the Naive (parametric) users in order to interact with database.

Casual Users / Temporary Users :

- Casual Users are the users who occasionally use/access the database but each time when they access the database they require the new information, for example, Middle or higher level manager.

Transaction Management

- Often, several operations on the database form a single logical unit of work. An example is a funds transfer, one account (say A) is debited and another account (say B) is credited. Clearly, it is essential that either both the credit and debit occur, or that neither occur. That is, the funds transfer must happen in its entirety or not at all. This all-or-none requirement is called atomicity.
- In addition, it is essential that the execution of the funds transfer preserve the consistency of the database. That is, the value of the sum $A + B$ must be preserved. This correctness requirement is called consistency.
- Finally, after the successful execution of a funds transfer, the new values of accounts A and B must persist, despite the possibility of system failure. This persistence requirement is called durability. **A transaction is a collection of operations that performs a single logical function in a database application.** Each transaction is a unit of both atomicity and consistency. Thus, we require that transactions do not violate any database-consistency constraints. That is, if the database was consistent when a transaction started, the database must be consistent when the transaction successfully terminates.
- Ensuring the atomicity and durability properties is the responsibility of the database system itself—specifically, of the transaction-management component. In the absence of failures, all transactions complete successfully, and atomicity is achieved easily. Finally, when several transactions update the database concurrently, the consistency of data may no longer be preserved, even though each individual transaction is correct. It is the responsibility of the concurrency-control manager to control the interaction among the concurrent transactions, to ensure the consistency of the database. Database systems designed for use on small personal computers may not have all these features.

Database Architecture:

- We are now in a position to provide a single picture (Figure 1.3) of the various components of a database system and the connections among them.
- The architecture of a database system is greatly influenced by the underlying computer system on which the database system runs.
- Database systems can be centralized, or client-server, where one server machine executes work on behalf of multiple client machines. Database systems can also be designed to exploit parallel computer architectures. Distributed databases span multiple geographically separated machines.

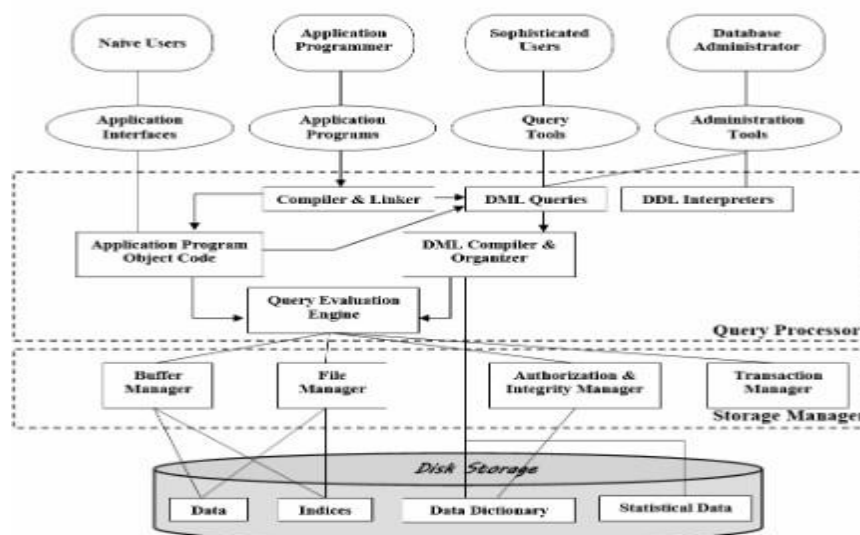


Figure: System Architecture

- A database system is partitioned into modules that deal with each of the responsibilities of the overall system. The functional components of a database system can be broadly divided into the **storage manager** and the query processor components.
- The storage manager is important because databases typically require a large amount of storage space.

- The query processor is important because it helps the database system simplify and facilitate access to data. **It is the job of the database system to translate updates and queries written in a nonprocedural language**, at the logical level, into an efficient sequence of operations at the physical level.
- Database applications are usually partitioned into two or three parts, as in Figure 1.4. In a two-tier architecture, the application resides at the client machine, where it invokes database system functionality at the server machine through query language statements.
- Application program interface standards like ODBC and JDBC are used for interaction between the client and the server. In contrast, in a three-tier architecture, the client machine acts as merely a front end and does not contain any direct database calls. Instead, the client end communicates with an application server, usually through a forms interface.
- The application server in turn communicates with a database system to access data. The business logic of the application, which says what actions to carry out under what conditions, is embedded in the application server, instead of being distributed across multiple clients.
- Three-tier applications are more appropriate for large applications, and for applications that run on the Worldwide Web.
- **Buffer Manager:** It is responsible for cache memory and the transfer of data between the secondary storage and main memory.

Query Processor:

- It interprets the requests (queries) received from end user via an application program into instructions. It also executes the user request which is received from the DML compiler.

Query Processor contains the following components –

- **DML Compiler:** It processes the DML statements into low level instruction (machine language), so that they can be executed.
- **DDL Interpreter:** It processes the DDL statements into a set of table containing meta data (data about data).
- **Embedded DML Pre-compiler:** It processes DML statements embedded in an application program into procedural calls.

Query Optimizer:

- It executes the instruction generated by DML Compiler.

Storage Manager:

- Storage Manager is a program that provides an interface between the data stored in the database and the queries received. It is also known as Database Control System. It maintains the consistency and integrity of the database by applying the constraints and executing the DCL statements. It is responsible for updating, storing, deleting, and retrieving data in the database.

It contains the following components –

Authorization Manager:

- It ensures role-based access control, i.e., checks whether the particular person is privileged to perform the requested operation or not.

Integrity Manager:

- It checks the integrity constraints when the database is modified.

Transaction Manager:

- It controls concurrent access by performing the operations in a scheduled way that it receives the transaction. Thus, it ensures that the database remains in the consistent state before and after the execution of a transaction.

File Manager:

- It manages the file space and the data structure used to represent information in the database.

Buffer Manager:

- It is responsible for cache memory and the transfer of data between the secondary storage and main memory.

Disk Storage: It contains the following components –

Data Files: It stores the data.

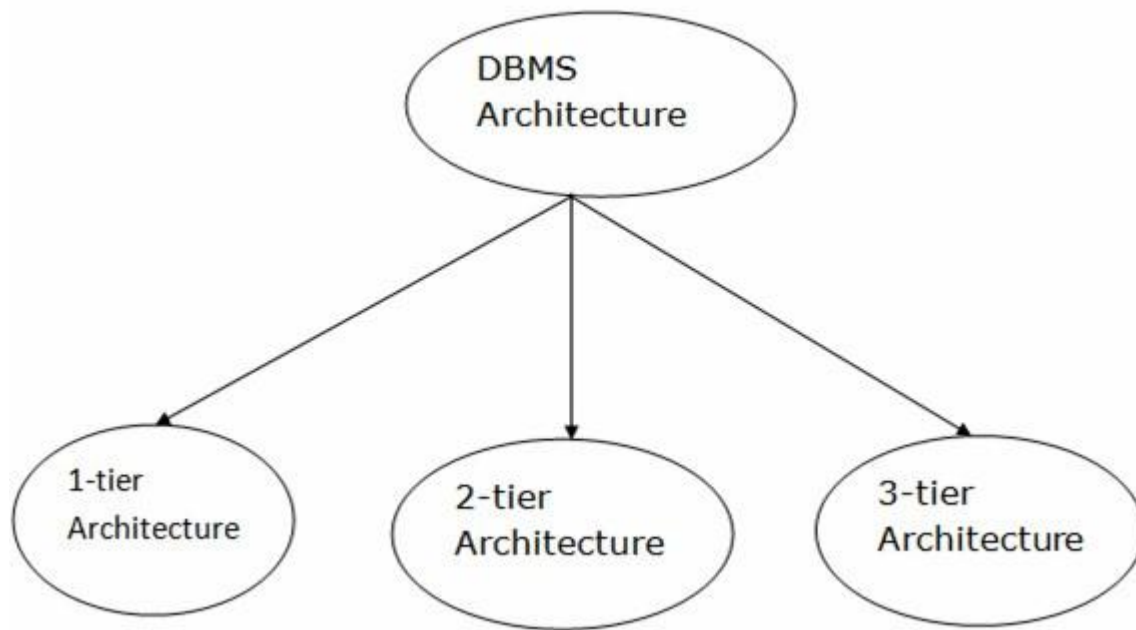
Data Dictionary: It contains the information about the structure of any database object. It is the repository of information that governs the metadata.

Indices: It provides faster retrieval of data item.

DBMS Architecture

- The DBMS design depends upon its architecture. The basic client/server architecture is used to deal with a large number of PCs, web servers, database servers and other components that are connected with networks.
- The client/server architecture consists of many PCs and a workstation which is connected via the network. DBMS architecture depends upon how users are connected to the database to get their request done.

Types of DBMS Architectures:



- Database architecture can be seen as a single tier or multi-tier. But logically, database architecture is of two types like: 2-tier architecture and 3-tier architecture.

1-Tier Architecture

- In this architecture, the database is directly available to the user. It means the user can directly sit on the DBMS and uses it.
- Any changes done here will directly be done on the database itself. It doesn't provide a handy tool for end users.
- The 1-Tier architecture is used for development of the local application, where programmers can directly communicate with the database for the quick response.

2-Tier Architecture

- The 2-Tier architecture is same as basic client-server. In the two-tier architecture, applications on the client end can directly communicate with the database at the server side. For this interaction, API's like: ODBC, JDBC are used.
- The user interfaces and application programs are run on the client-side.
- The server side is responsible to provide the functionalities like: query processing and transaction management.
- To communicate with the DBMS, client-side application establishes a connection with the server side.

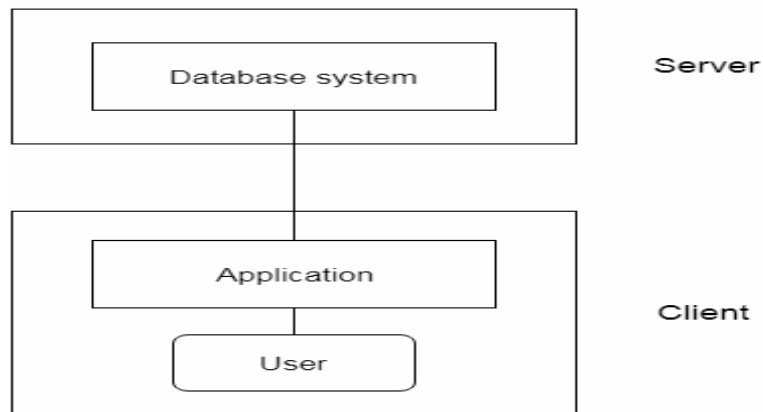


Fig: 2-tier Architecture

3-Tier Architecture

- The 3-Tier architecture contains another layer between the client and server. In this architecture, client can't directly communicate with the server.
- The application on the client-end interacts with an application server which further communicates with the database system.
- End user has no idea about the existence of the database beyond the application server. The database also has no idea about any other user beyond the application.
- The 3-Tier architecture is used in case of large web application.

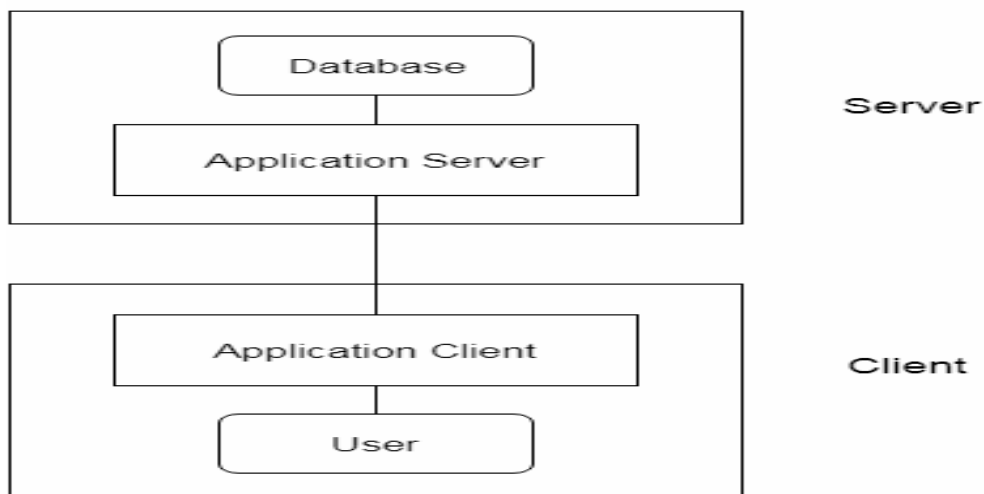


Fig: 3-tier Architecture

Conceptual Database Design

Database design and ER model

- Databases were deployed much more extensively than ever before. Database systems also had to **support Web interfaces to data**.
- The database design process can be divided into **six steps**. **The ER model is most relevant to the first three steps**

Requirements Analysis:

- The very first step in designing a database application is to understand what data is to be stored in the database, what applications must be built on top of it, and what operations are most frequent and subject to performance requirements. In other words, we must find out what the users want from the database. This is usually an informal process that involves discussions with user groups, a study of the current operating environment and how it is expected to change, analysis of any available documentation on existing applications that are expected to be replaced or complemented by the database, and so on. **Several methodologies have been proposed for organizing and presenting the information gathered in this step, and some automated tools have been developed to support this process.**

Conceptual Database Design:

- The information gathered in the requirements analysis step is used to develop a **high-level description of the data** to be stored in the database, along with the constraints that are known to hold over this data. **This step is often carried out using the ER model**, or a similar high-level data model, and is discussed in the rest of this chapter.

Logical Database Design:

- We must choose a DBMS to implement our database design, and convert the conceptual database design into a database schema in the data model of the chosen DBMS. We will only consider relational DBMSs, and therefore, the task in the logical design step is to convert an ER schema into a relational database schema. We discuss this step in detail in Chapter 3; the result is a conceptual schema, sometimes called the logical schema, in the relational data model. The entity-relationship (ER) data model allows us to describe the data involved in a real-world enterprise in terms of objects and their relationships and is widely used to develop an initial database design. The ER model is important primarily for its role in database design. It provides useful concepts that allow us to move from an informal description of what users want from their database to a more detailed and precise, description that can be implemented in a DBMS

Beyond the ER Model

- ER modeling is sometimes regarded as a complete approach to designing a logical database schema. This is incorrect because the ER diagram is just an approximate description of the data, constructed through a very subjective evaluation of the information collected during requirements analysis. A more careful analysis can often refine the logical schema obtained at the end of Step 3. Once we have a good logical schema, we must consider performance criteria and design the physical schema. Finally, we must address security issues and ensure that users are able to access the data they need, but not data that we wish to hide from them. The remaining three steps of database design are briefly described below:

Schema Refinement:

- The fourth step in database design is to analyze the collection of relations in our relational database schema to **identify potential problems**, and to refine it. In contrast to the requirements analysis and conceptual design steps, which are essentially subjective, schema refinement can be guided by some elegant and powerful theory. We discuss the theory of normalizing relations, restructuring them to ensure some desirable properties.

Physical Database Design:

- In this step we must consider typical expected workloads that our database must support and further refine the database design to ensure that it meets desired performance criteria. This step may simply involve building indexes on some tables and clustering some tables, or it may involve a substantial design of parts of the database schema obtained from the earlier design steps

Security Design:

- In this step, we identify different user groups and different roles played by various users (e.g., the development team for a product, the customer support representatives, and the product manager). For each role and user group, we must identify the parts of the database that they must be able to access and the parts of the database that they should not be allowed to access, and take steps to ensure that they can access only the necessary parts. A DBMS provides several mechanisms to assist in this step

Entity Relationship(ER) Modeling:

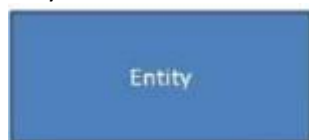
- Database Design Techniques
 1. ER Modeling (Top down Approach)
 2. Normalization (Bottom Up approach)

What is ER Modeling?

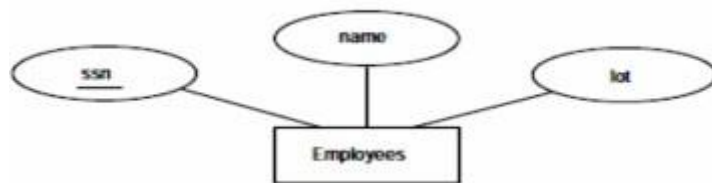
- A graphical technique for understanding and organizing the data independent of the actual database implementation
- An ER model stands for the Entity-Relationship model that **Peter Chen developed in 1976. This model consists of a collection of entities (Real word objects) and their relationships.** It describes the database's **conceptual view**. We must ensure that no two entities are identical in this context.
- ER Model describes the system's logical view from a data perspective formed by the entity set, relationship set, and attributes. In this model, all entities come under the entity set, all relations between the entities come under the relationship set, and attributes describe the properties of entities.
- **Let us understand ER model with an example. Suppose we have two real-world entities named Student and Branch that will further form an Entity set.** Now we can easily form a relation between them as the **Student belongs to a Branch**. It shows how we can get a relationship set from ER Model. Noted that the ER Model content must conform to constraints like mapping cardinality. Finally, the attributes of these entities would be:
- We need to be familiar with the following terms to go further.

Entity

- **An entity is an object in the real world that is distinguishable from other objects.** It is often useful to identify a collection of similar entities. Such a collection is called an entity set. Note that entity sets need not be disjoint; An entity is described using a set of attributes. All entities in a given entity set have the same attributes; this is essentially what we mean by similar. For each attribute associated with an entity set, we must identify a domain of possible values, for each entity set, we choose a key
- A key is a minimal set of attributes whose values uniquely identify an entity in the set. There could be more than one candidate key; if so, we designate one of them as the primary key. For now we will assume that each entity set contains at least one set of attributes that uniquely identifies an entity in the entity set; that is, the set of attributes contains a key.



Example:



Entity instance

- Entity instance is a particular member of the entity type.
Example for entity instance: A particular employee

Regular Entity

- An entity which has its own key attribute is a regular entity.
Example for regular entity: **Employee**.

Weak entity

- An entity which depends on other entity for its existence and doesn't have any key attribute of its own is a weak entity.
Example for a weak entity: In a parent/child relationship, a parent is considered as a strong entity and the child is a weak entity.
In ER modeling, notation for weak entity is given below.



Attributes

- Properties/characteristics which describe entities are called attributes.
In ER modeling, notation for attribute is given below.



Domain of Attributes

- **The set of possible values that an attribute can take is called the domain of the attribute.**
- For example, the attribute day may take any value from the set {Monday, Tuesday ... Friday}. Hence this set can be termed as the domain of the attribute day.

Key attribute

- The attribute (or combination of attributes) which is unique for every entity instance is called key attribute.
- E.g the employee_id of an employee, pan_card_number of a person etc. If the key attribute consists of two or more attributes in combination; it is called a composite key.
In ER modeling, notation for key attribute is given below



Simple attribute

- If an attribute cannot be divided into simpler components, it is a simple attribute.
Example for simple attribute: employee_id of an employee.

Composite attribute

- If an attribute can be split into components, it is called a composite attribute.
Example for composite attribute: Name of the employee which can be split into First_name, Middle_name, and Last_name.

Single valued Attributes

- If an attribute can take only a single value for each entity instance, it is a single valued attribute.
Example for single valued attribute: age of a student. It can take only one value for a particular student.

Multi-valued Attributes

- If an attribute can take more than one value for each entity instance, it is a multi-valued attribute. Multi-valued
Example for multi valued attribute: telephone number of an employee, a particular employee may have multiple telephone numbers.

In ER modeling, notation for multi-valued attribute is given below.



Stored Attribute

- An attribute which need to be stored permanently is a stored attribute
Example for stored attribute: name of a student

Derived Attribute

- An attribute which can be calculated or derived based on other attributes is a derived attribute.
Example for derived attribute: age of employee which can be calculated from date of birth and current date.
In ER modeling, notation for derived attribute is given below.



Relationships

- Associations between entities are called relationships

Example: An employee works for an organization. Here "works for" is a relation between the entities employee and organization.

In ER modeling, notation for relationship is given below



However in ER Modeling, to connect a weak Entity with others, you should use a weak relationship notation as given below



Degree of a Relationship

- It is the number of entity set associated or participated in the relationship set of an E-R Diagram. The n-ary relationship is the general form for degree n. Special cases are unary, binary, and ternary, where the degree is 1, 2, and 3, respectively.

Example for unary relationship: An employee is a manager of another employee

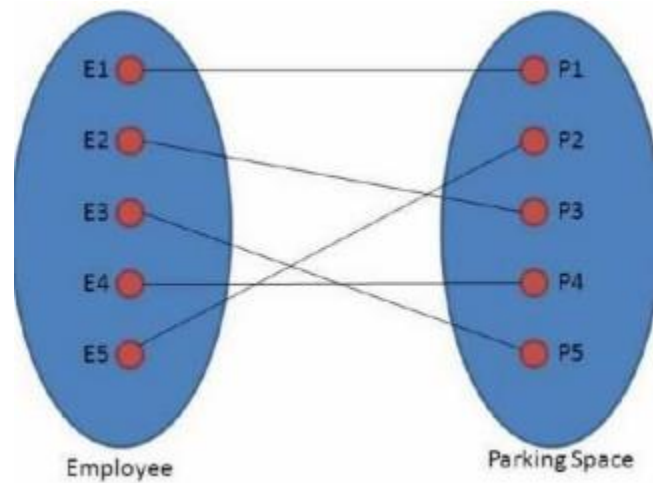
Example for binary relationship: An employee works-for department.

Example for ternary relationship: customer purchase item from a shop keeper

Cardinality of a Relationship

- It represents the number of entities to which another entity can be associated through a relationship set in E-R Diagram
- Relationship cardinalities specify how many of each entity type is allowed. Relationships can have four possible connectivity's as given below.
 1. One to one (1:1) relationship
 2. One to many (1:N) relationship
 3. Many to one (M:1) relationship
 4. Many to many (M:N) relationship
- The minimum and maximum values of this connectivity is called the cardinality of the relationship

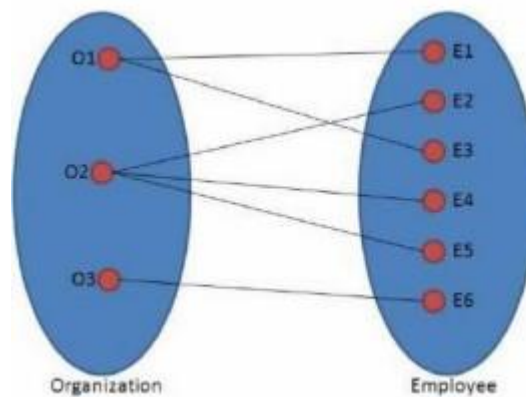
Example for Cardinality – One-to-One (1:1)
Employee is assigned with a parking space.



- One employee is assigned with only one parking space and one parking space is assigned to only one employee. Hence it is a 1:1 relationship and cardinality is One-To-One (1:1)
In ER modeling, this can be mentioned using notations as given below



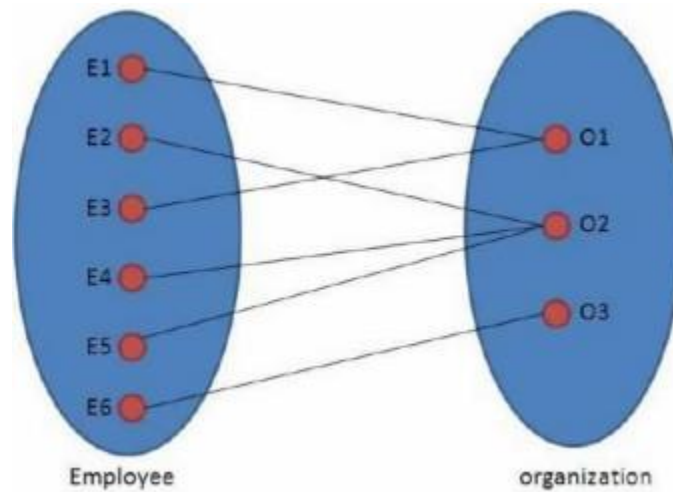
Example for Cardinality – One-to-Many (1:N)
Organization has employees



- One organization can have many employees, but one employee works in only one organization. Hence it is a 1:N relationship and cardinality is One-To-Many (1:N)
In ER modeling, this can be mentioned using notations as given below



Example for Cardinality – Many-to-One (M :1)
It is the reverse of the One to Many relationships. Employee works in organization

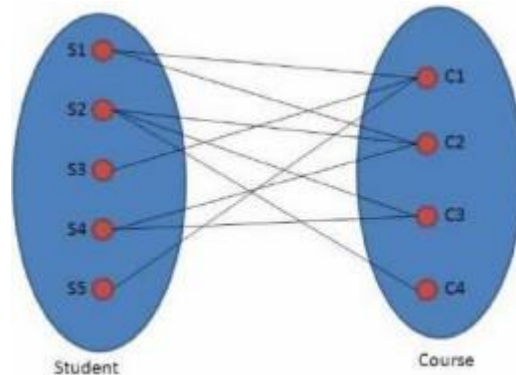


- One employee works in only one organization But one organization can have many employees. Hence it is a M:1 relationship and cardinality is Many-to-One (M :1)

In ER modeling, this can be mentioned using notations as given below.



Cardinality – Many-to-Many (M:N) Students enrolls for courses



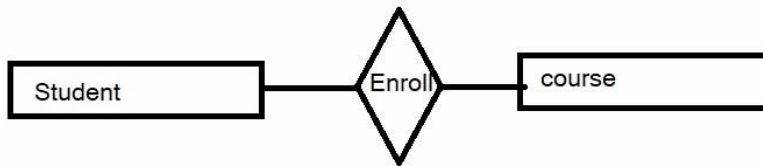
- One student can enroll for many courses and one course can be enrolled by many students. Hence it is a M:N relationship and cardinality is Many-to-Many (M:N) In ER modeling, this can be mentioned using notations as given below



Relationship:

=====

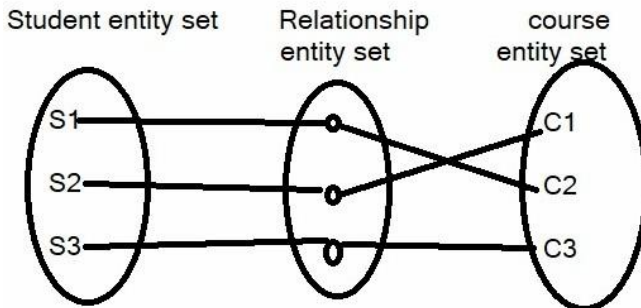
- A relationship is defined as an association among several entities
- Relationships are represented by diamond symbol



Relationship Sets:

=====

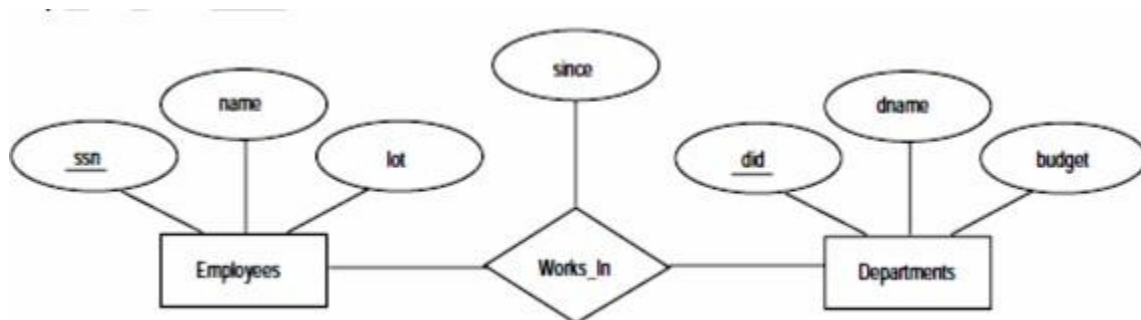
- A set of relationships of the same type
- A relationship can be thought of as a set of n-tuples
- $\{(e1.....en)/e1 \in E1,.....,en \in En\}$



Relationship contain attributes: (primary key (e1) U primary key(e2)Uprimary key(en))+descriptive attributes

Descriptive Attribute:

=====



- Descriptive attributes are used to record information about relationship
- Descriptive attributes are attributes of relationship
- From the above diagram since is the descriptive attribute

Relationship Participation

1. Total

- In total participation, every entity instance will be connected through the relationship to another instance of the other participating entity types

2. Partial

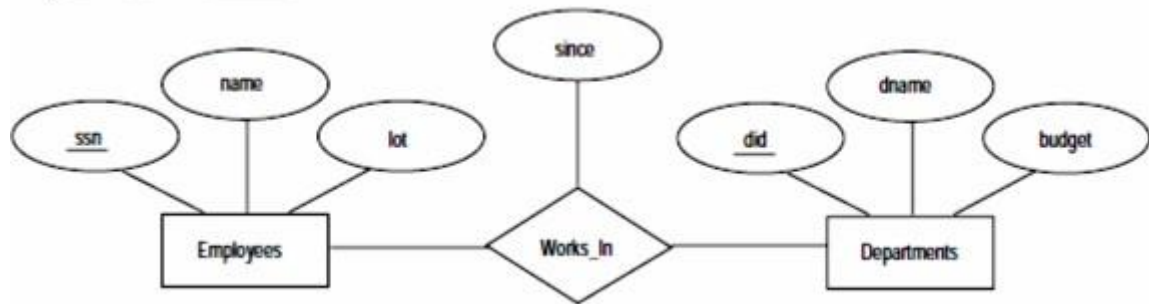
- Example for relationship participation
 - Consider the relationship - Employee is head of the department.
 - Here all employees will not be the head of the department. Only one employee will be the head of the department. In other words, only few instances of employee entity participate in the above relationship. So employee entity's participation is partial in the said relationship.
 - However each department will be headed by some employee. So department entity's participation is total in the said relationship.

RELATIONSHIPS AND RELATIONSHIP SETS

- A relationship is an association among two or more entities. As with entities, we may wish to collect a set of similar relationships into a relationship set.
- A relationship set can be thought of as a set of n-tuples:

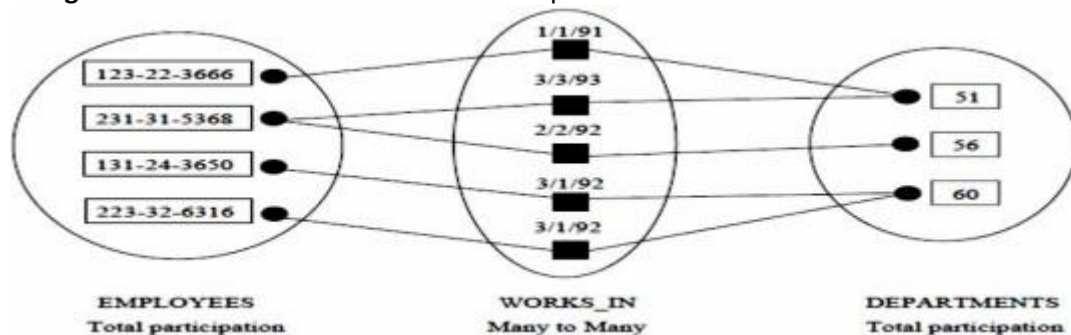
○ $\{(e_1, \dots, e_n) \mid e_1 \in E_1, \dots, e_n \in E_n\}$

- Each n-tuple denotes a relationship involving n entities e_1 through e_n , where entity e_i is in entity set E_i . In the following diagram, we show the relationship set Works In, in which each relationship indicates a department in which an employee works.
- Note that several relationship sets might involve the same entity sets. For example, we could also have a Manages relationship set involving Employees and Departments



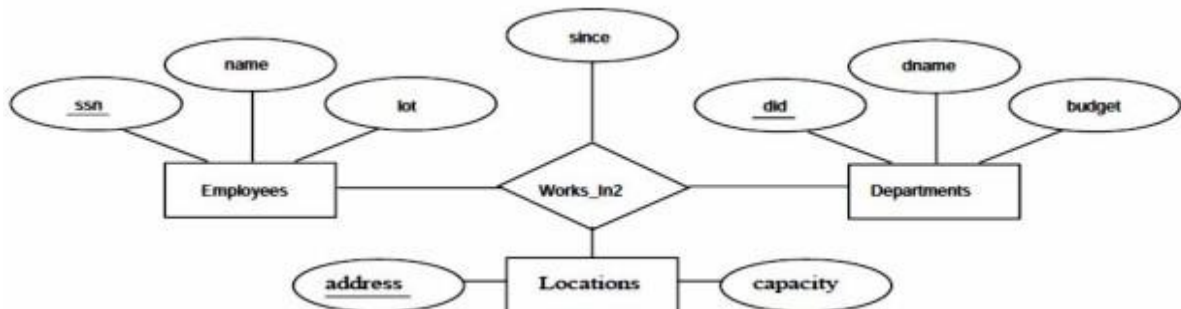
- A relationship can also have **descriptive attributes**. **Descriptive attributes are used to record information about the relationship**, rather than about any one of the participating entities; for example, we may wish to record that Attishoo works in the pharmacy department as of January 1991. This information is captured in Figure 2.2 by adding an attribute, since, to Works In. A relationship must be uniquely identified by the participating entities, without reference to the descriptive attributes. In the Works In relationship set, for example, each Works In relationship must be uniquely identified by the combination of employee ssn and department did. Thus, for a given employee-department pair, we cannot have more than one associated since value. An instance of a relationship set is a set of relationships. Intuitively, an instance can be thought of as a 'snapshot' of the relationship set at some instant in time. Each Employees entity is denoted by its ssn, and each Departments entity is denoted by its did, for simplicity. The since value is shown beside each relationship.

Fig: An instance of the works in relationship set



- As another example of an ER diagram, suppose that each department has offices in several locations and we want to record the locations at which each employee works. This relationship is ternary because we must record an association between an employee, a department, and a location.

Fig: A Ternary Relationship set



ADDITIONAL FEATURES OF THE ER MODEL

We now look at some of the constructs in the ER model that allow us to describe some subtle properties of the data. The expressiveness of the ER model is a big reason for its wide spread use.

Key Constraints

Consider the Works In relationship shown in following picture An employee can work in several departments, and a department can have several employees, as illustrated in the Works In instance shown in Figure Employee 231-31-5368 has worked in Department 51 since 3/3/93 and in Department 56 since 2/2/92. Department 51 has two employees

Now consider another relationship set called Manages between the Employees and Departments entity sets such that each department has at most one manager, although a single employee is allowed to manage more than one department. The restriction that each department has at most one manager is an example of a key constraint, and it implies that each Departments entity appears in at most one Manages relationship in any allowable instance of Manages. This restriction is indicated in the ER diagram of Figure 2.6 by using an arrow from Departments to Manages. Intuitively, the arrow states that given a Departments entity, we can uniquely determine the Manages relationship in which it appears

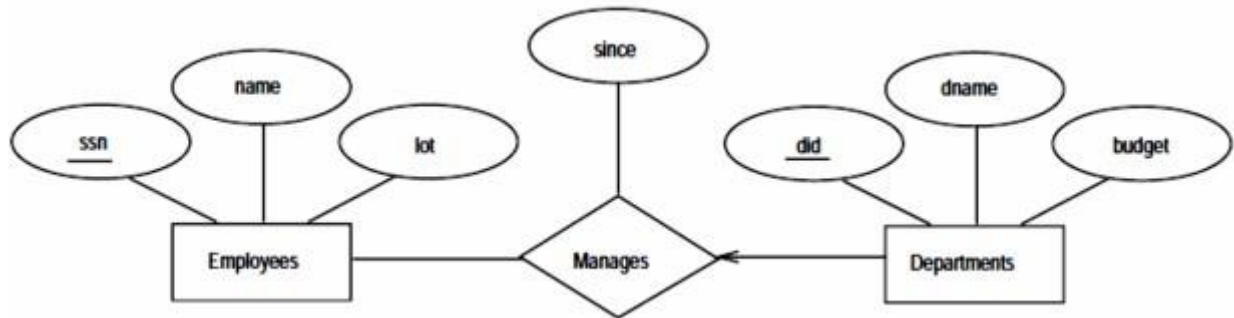


Fig: Key constraint on manages

- An instance of the Manages relationship set is shown in Figure 2.7. While this is also a potential instance for the Works In relationship set, the instance of Works In shown in Figure 2.3 violates the key constraint on Manages.

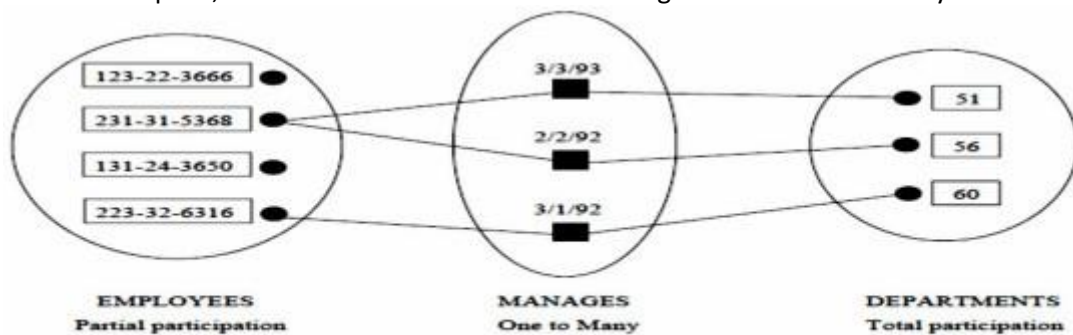


Figure 2.7 An Instance of the Manages Relationship Set

- A relationship set like Manages is sometimes said to be one-to-many, to indicate that one employee can be associated with many departments (in the capacity of a manager), whereas each department can be associated with at most one employee as its manager. In contrast, the Works In relationship set, in which an employee is allowed to work in several departments and a department is allowed to have several employees, is said to be many-to-many.

Key Constraints for Ternary Relationships

- We can extend this convention and the underlying key constraint concept to relationship sets involving three or more entity sets: If an entity set E has a key constraint in a relationship set R, each entity in an instance of E appears in at most one relationship in (a corresponding instance of) R. To indicate a key constraint on entity set E in relationship set R, we draw an arrow from E to R. In Figure 2.8, we show a ternary relationship with key constraints. Each employee works in at most one department, and at a single location.

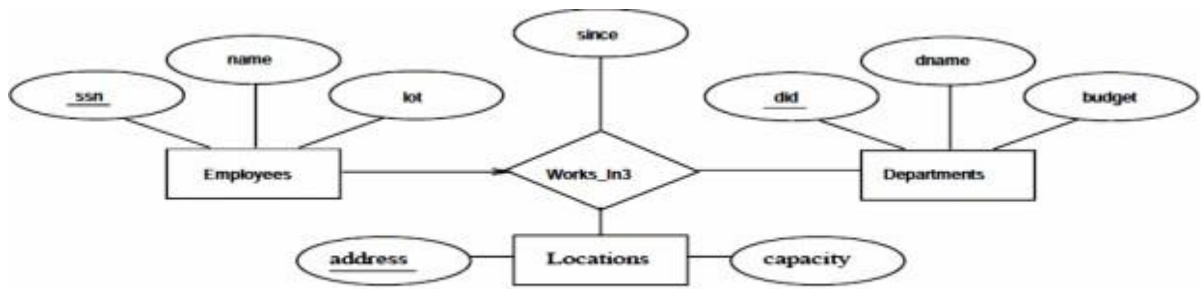


Figure 2.8 A Ternary Relationship Set with Key Constraints

- An instance of the Works In relationship set is shown in Figure 2.9. Notice that each department can be associated with several employees and locations, and each location can be associated with several departments and employees; however, each employee is associated with a single department and location

Participation Constraints

- it specifies whether the existence of an entity depends on its being related to another entity via a relationship set
- these constraint ell
 - **Minimum Cardinality**
 - the minimum number of entities to which an entity must relate through a relationship set
 - **Maximum Cardinality**
 - the maximum number of entities to which an entity must relate through a relationship set
- The key constraint on Manages tells us that a department has at most one manager .A natural question to ask is whether every department has a manager. Let us say that every department is required to have a manager. This requirement is an example of a participation constraint; the participation of the entity set Departments in the relationship set Manages is said to be total. A participation that is not total is said to be partial. As an example, the participation of the entity set Employees in Manages is partial, since not every employee gets to manage a department. Revisiting the Works In relationship set, it is natural to expect that each employee works in at least one department and that each department has at least one employee. This means that the participation of both Employees and Departments in Works In is total. The ER diagram in Figure 2.10 shows both the Manages and Works In relationship sets and all the given constraints. If the participation of an entity set in a relationship set is total, the two are connected by a thick line; independently, the presence of an arrow indicates a key constraint. The instances of Works In and Manages shown in Figures 2.3 and 2.7 satisfy all the constraints in Figure 2.10

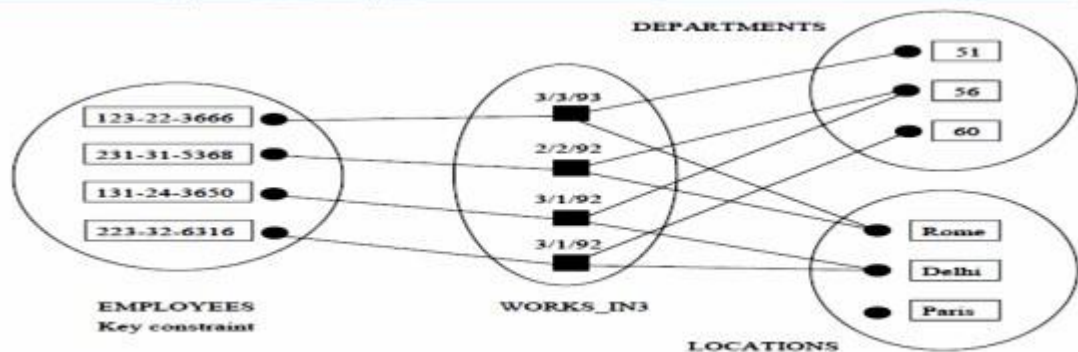
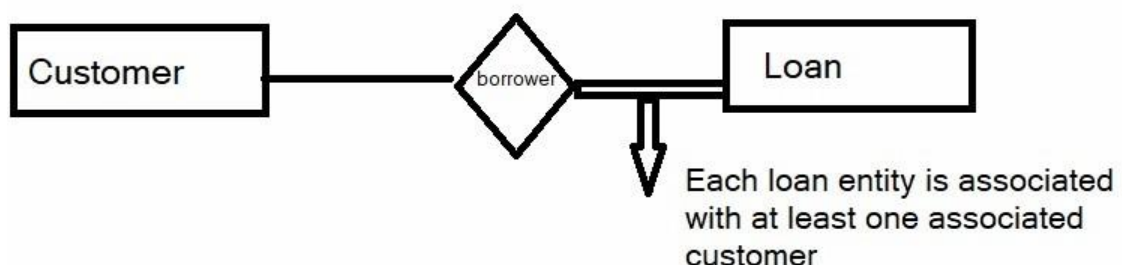


Figure 2.9 An Instance of Works_In3

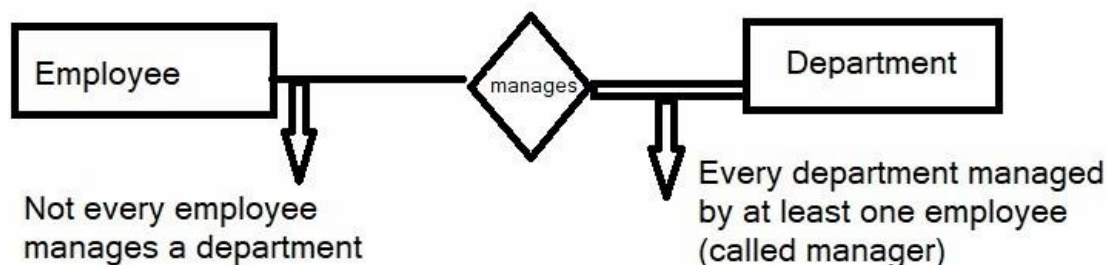
Total Participation:

- Each entity in the entity set occurs in at least one relationship in the relationship set



Partial Participation:

- Each entity in the entity set may not occur in at least one relationship in that relationship set



Weak Entities

- Thus far, we have assumed that the attributes associated with an entity set include a key. This assumption does not always hold. For example, suppose that employees can purchase insurance policies to cover their dependents. We wish to record information about policies, including who is covered by each policy, but this information is really our only interest in the dependents of an employee. If an employee quits, any policy owned by the employee is terminated and we want to delete all the relevant policy and dependent information from the database. We might choose to identify a dependent by name alone in this situation, since it is reasonable to expect that the dependents of a given employee have different names. Thus the attributes of the Dependents entity set might be pname and age. The attribute pname does not identify a dependent uniquely

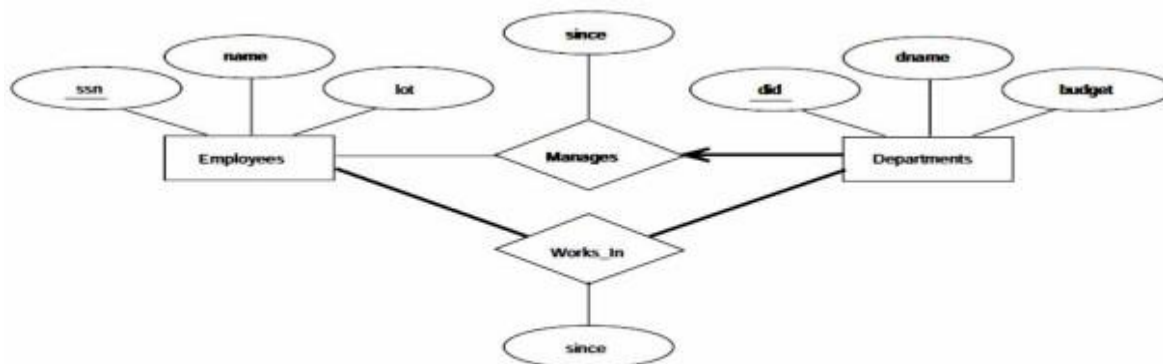


Figure 2.10 Manages and Works_In

- Dependents are an example of a weak entity set. A weak entity can be identified uniquely only by considering some of its attributes in conjunction with the primary key of another entity, which is called the identifying owner. The following restrictions must hold: The owner entity set and the weak entity set must participate in a one-to-many relationship set (one owner entity is associated with one or more weak entities, but each weak entity has a single owner). This relationship set is called the identifying relationship set of the weak entity set. The weak entity set must have total participation in the identifying relationship set.

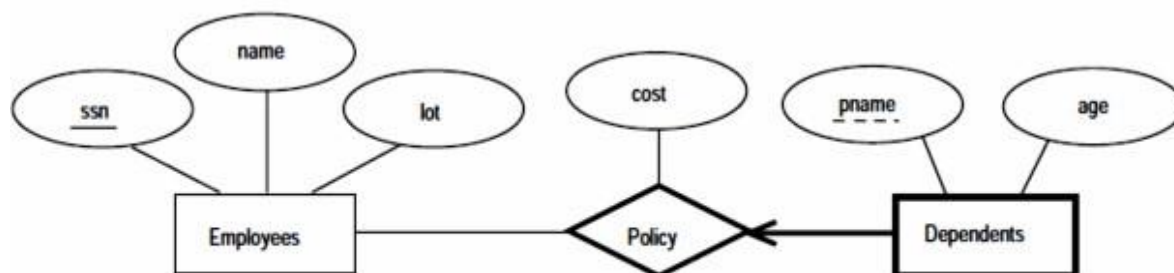


Figure 2.11 A Weak Entity Set

Class Hierarchies

- Sometimes it is natural to classify the entities in an entity set into subclasses. Forexample, we might want to talk about an Hourly Emps entity set and a Contract Emps entity set to distinguish the basis on which they are paid. We might have attributes hours worked and hourly wage defined for Hourly Emps and an attribute contracted defined for Contract Emps. We want the semantics that every entity in one of these sets is also an Employees

entity, and as such must have all of the attributes of Employees defined. Thus, the attributes defined for an Hourly Emps entity are the attributes for Employees plus Hourly Emps. We say that the attributes for the entity set Employees are inherited by the entity set Hourly Emps, and that Hourly Emps ISA (read is a) Employees. In addition, and in contrast to class hierarchies in programming languages such as C++, there is a constraint on queries over instances of these entity sets: A query that asks for all Employees entities must consider all Hourly Emps

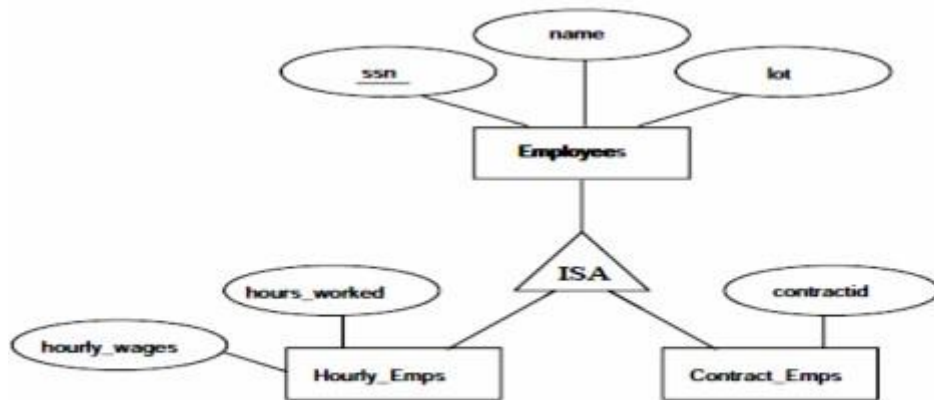


Figure 2.12 Class Hierarchy

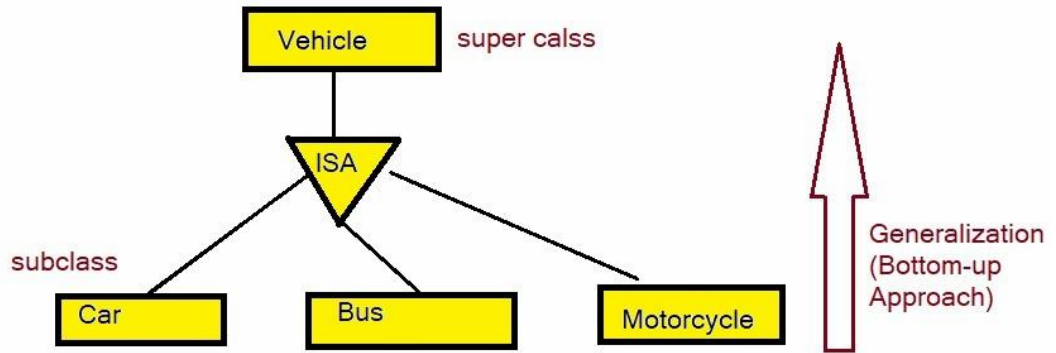
- A class hierarchy can be viewed in one of two ways: Employees is specialized into subclasses. Specialization is the process of identifying subsets of an entity set (the superclass) that share some distinguishing characteristic. Typically the superclass is defined first, the subclasses are defined next, and subclass-specific attributes and relationship sets are then added. Hourly Emps and Contract Emps are generalized by Employees. As another example, two entity sets Motorboats and Cars may be generalized into an entity set Motor Vehicles.

Extended Entity Relationship Features:

- As the complexity of data increased in the late 1980's, it became more and more difficult to use the traditional ER Model for database modeling. Hence some improvements or enhancements were made to the existing ER Model to make it able to handle the complex application better.
- Hence, as part of the extended ER Model, along with other improvements three new concepts were added to the existing ER Model.
 - Generalization
 - Specialization
 - Aggregation

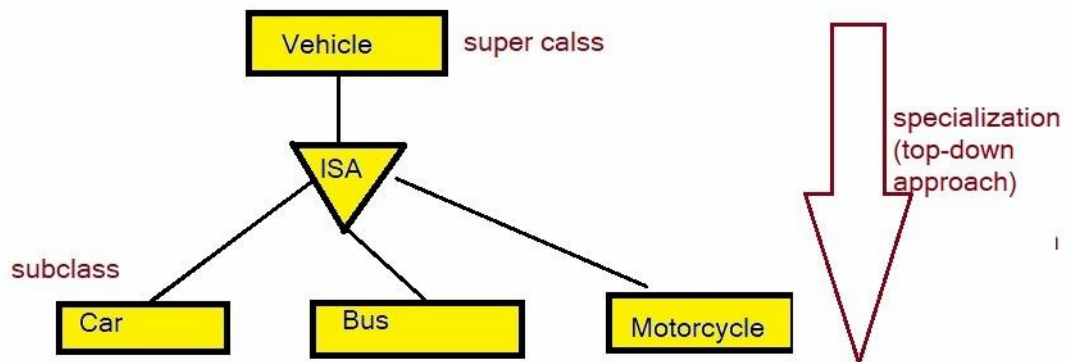
Generalization

- Consists of identifying some common characteristics of a collection of entity sets and creating a new entity set that contains entities possessing these common characteristics. Typically the subclasses are defined first, the super class is defined next, and any relationship sets that involve the super class are then defined. We can specify two kinds of constraints with respect to ISA hierarchies, namely, overlap and covering constraints. Overlap constraints determine whether two subclasses are allowed to contain the same entity. Covering constraints determine whether the entities in the subclasses collectively include all entities in the super class.
 - Example
 - Consider we have 3 sub entities Car, Bus and Motor Cycle. Now these three entities can be generalized into one higher-level entity (or Super class) named as vehicle



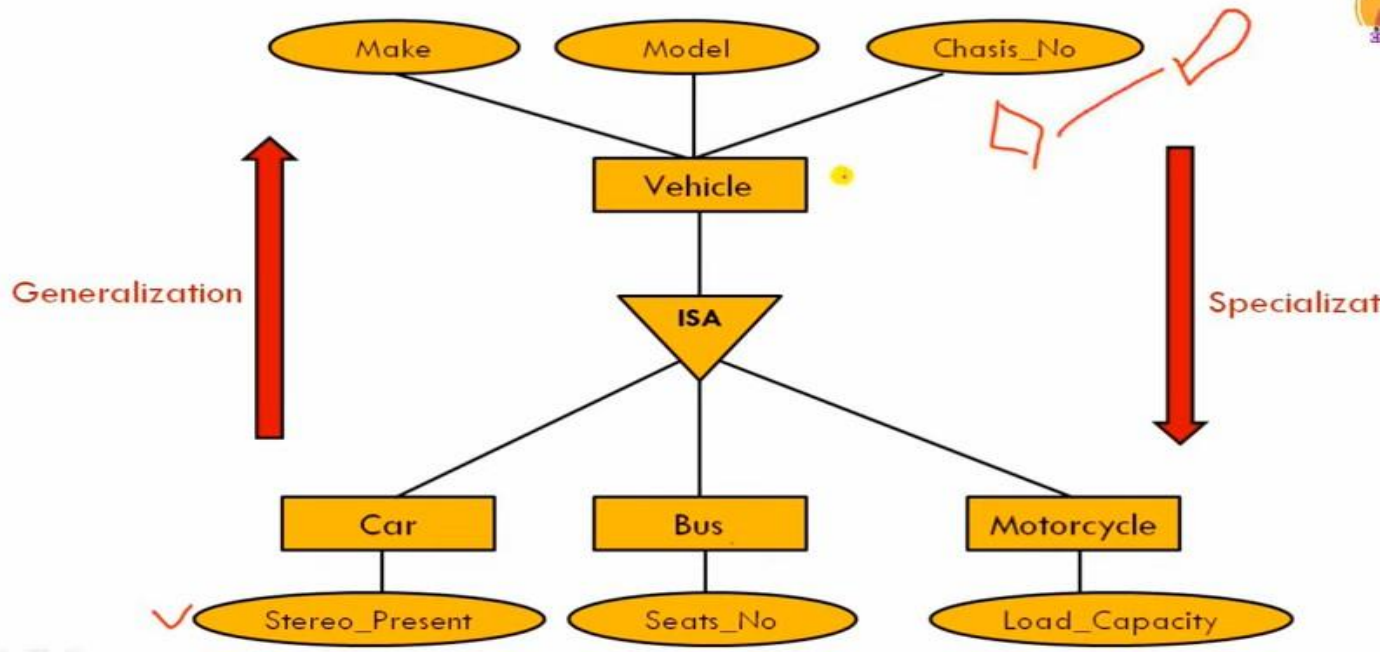
Specialization:

- Specialization is opposite to generalization
- In specialization, an entity is broken down into sub entities based on their characteristics.
- Specialization is a “top-down approach” where high level entity is specialized into two or more level entities
- Specialization is used to identify the subset of an entity set that share some distinguishing characteristics
- Specialization can be repeatedly applied to refine the design of schema
- Depicted by triangle component labeled ISA
 - Vehicle entity can be a Car, Truck or Motor Cycle
 - Normally the super class is defined first, the subclass and its related attributes are defined next, and relationship set are then added

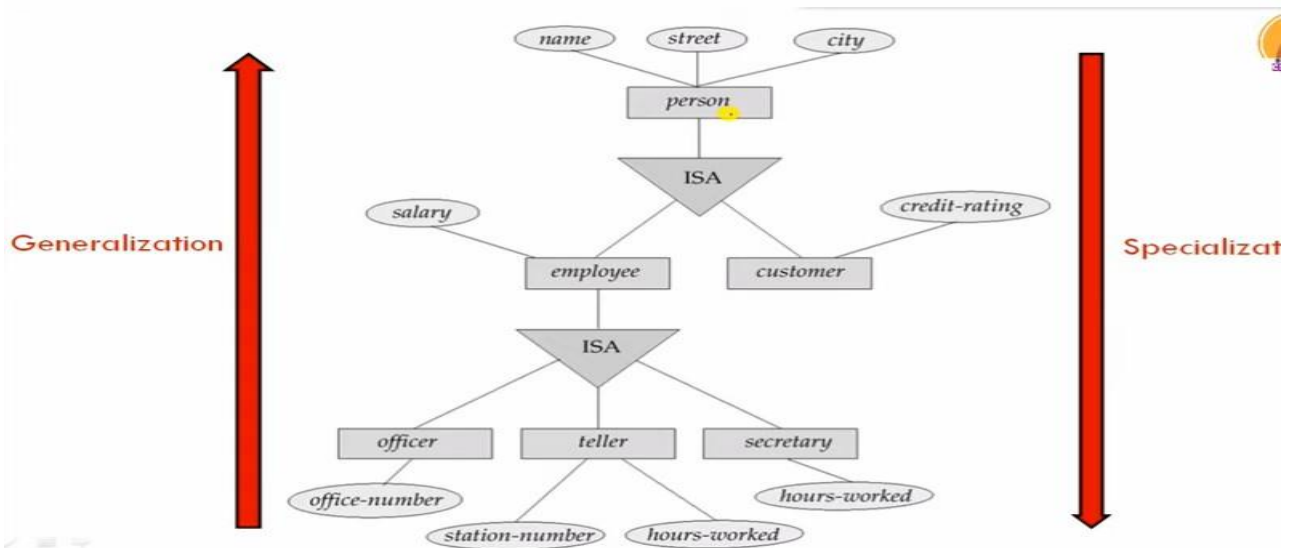


Inheritance:

- Inheritance is an importance feature of generalization and specialization
- Attribute inheritance allows lower level entities to inherit the attributes of higher level entities
 - For example consider relations of car and bus inheriting the attributes of vehicle. Thus car described by attributes of super class vehicle as well as its own attributes
- This also extends to participation inheritance in which relationships involving higher level entity sets are also inherited by lower level entity sets
 - A lower level entity set can participate in its own relationship sets too

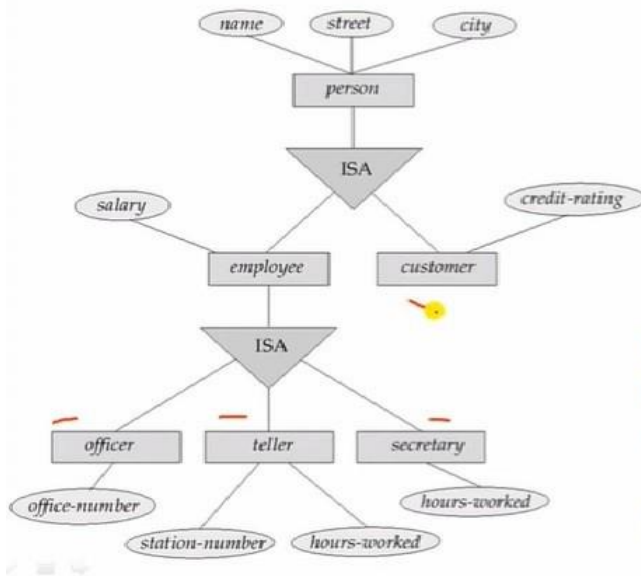


Example2:



Example:

How Schema or Tables can be formed?



Four tables can be formed:

1. **customer** (name, street, city, credit_rating)
2. **officer** (name, street, city, salary, office_number)
3. **teller** (name, street, city, salary, station_number, hours_worked)
4. **secretary** (name, street, city, salary, hours_worked)

Aggregation

- As we have defined it thus far, a relationship set is an association between entity sets. Sometimes we have to model a relationship between a collection of entities and relationships. Suppose that we have an entity set called Projects and that each Projects entity is sponsored by one or more departments. The Sponsors relationship set captures this information. A department that sponsors a project might assign employees to monitor the sponsorship. Intuitively, Monitors should be a relationship set that associates a Sponsors relationship (rather than a Projects or Departments entity) with an Employees entity. However, we have defined relationships to associate two or more entities. In order to define a relationship set such as Monitors, we introduce a new feature of the ER model, called aggregation. Aggregation allows us to indicate that a relationship set (identified through a dashed box) participates in another relationship set
- Aggregation is used when we need to express a relationship among relationships
- Aggregation is an abstraction through which relationships are treated as higher level entities
- Aggregation is a process of when a relationship between two entities is considered as single entity and again this single entity has relationship with another entity

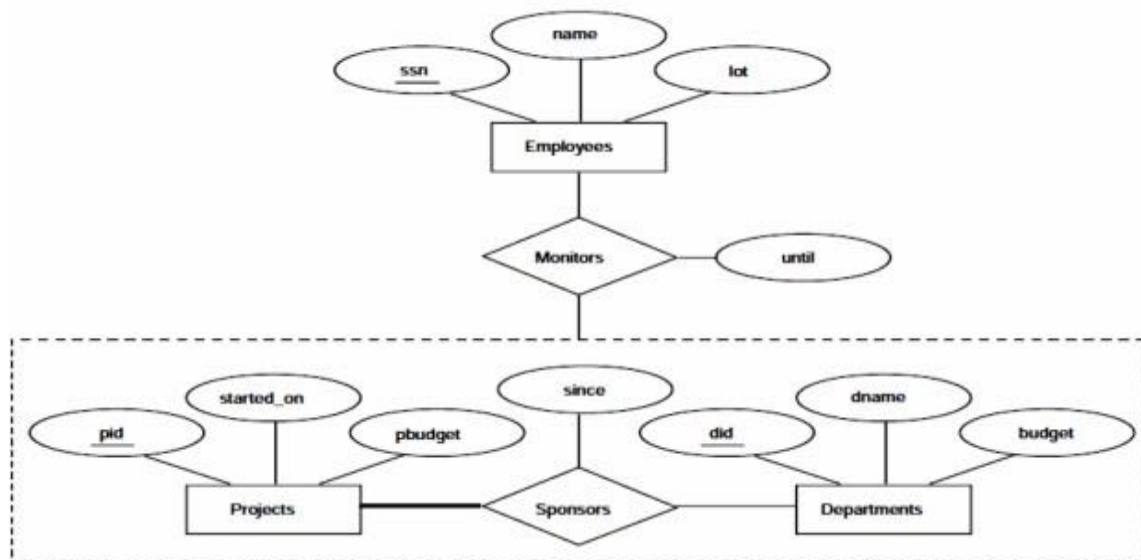


Figure 2.13 Aggregation

CONCEPTUAL DATABASE DESIGN WITH THE ER MODEL

- Developing an ER diagram presents several choices, including the following: Should a concept be modeled as an entity or an attribute?
 - Should a concept be modeled as an entity or a relationship?
 - What are the relationship sets and their participating entity sets? Should we use binary or ternary relationships? Should we use aggregation?

Entity versus Attribute

- While identifying the attributes of an entity set, it is sometimes not clear whether a property should be modeled as an attribute or as an entity set (and related to the first entity set using a relationship set). For example, consider adding address information to the Employees entity set. One option is to use an attribute address. This option is appropriate if we need to record only one address per employee, and it suffices to think of an address as a string. An alternative is to create an entity set called Addresses and to record associations between employees and addresses using a relationship (say, Has Address). This more complex alternative is necessary in two situations: We have to record more than one address for an employee. We want to capture the structure of an address in our ER diagram. For example, we might break down an address into city, state, country, and Zip code, in addition to a string for street information. By representing an address as an entity with these attributes, we can support queries such as "Find all employees with an address in Madison, WI."

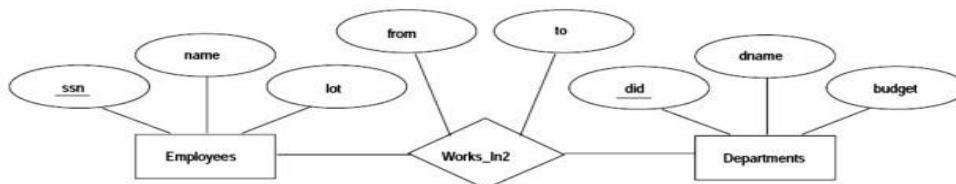


Figure 2.14 The Works_In2 Relationship Set

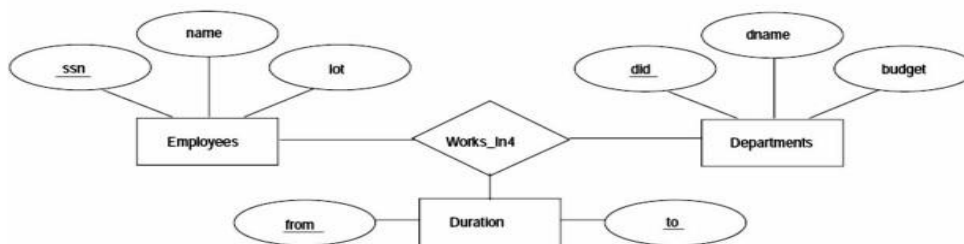


Figure 2.15 The Works_In4 Relationship Set

Entity versus Relationship

- Consider the relationship set called Manages, suppose that each department manager is given a discretionary budget (dbudget), as shown in Figure 2.16, in which we have also renamed the relationship set to Manages2.

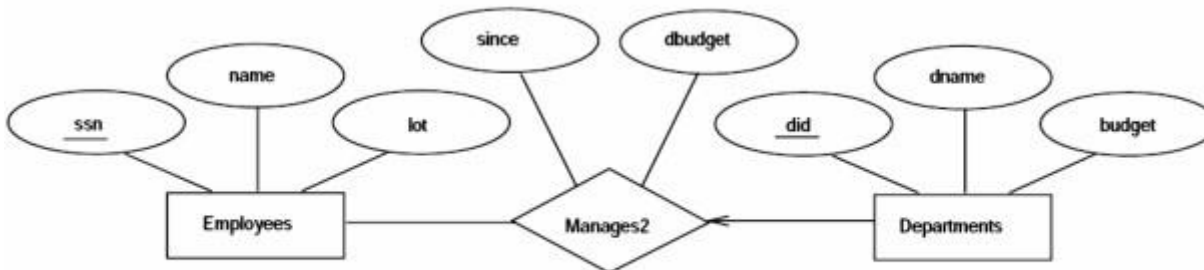


Figure 2.16 Entity versus Relationship

- We can address these problems by associating dbudget with the appointment of the employee as manager of a group of departments. In this approach, we model the appointment as an entity set, sayMgrAppt, and use a ternary relationship, say Manages3, to relate a manager, an appointment, and a department. The details of an appointment (such as the discretionary budget) are not repeated for each department that is included in the appointment now, although there is still one Manages3 relationship instance per such department

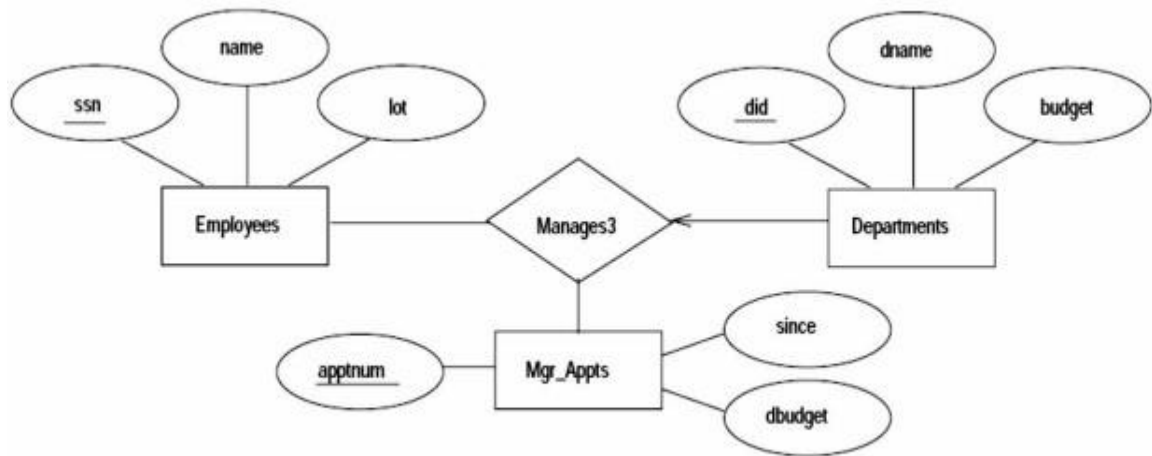


Figure 2.17 Entity Set versus Relationship

Binary versus Ternary Relationships *

- Consider the ER diagram shown in Figure 2.18. It models a situation in which an employee can own several policies, each policy can be owned by several employees, and each dependent can be covered by several policies. Suppose that we have the following additional requirements: 1. A policy cannot be owned jointly by two or more employees. 2. Every policy must be owned by some employee. 3. Dependents is a weak entity set, and each dependent entity is uniquely identified by taking pname in conjunction with the policyid of a policy entity (which, intuitively, covers the given dependent).

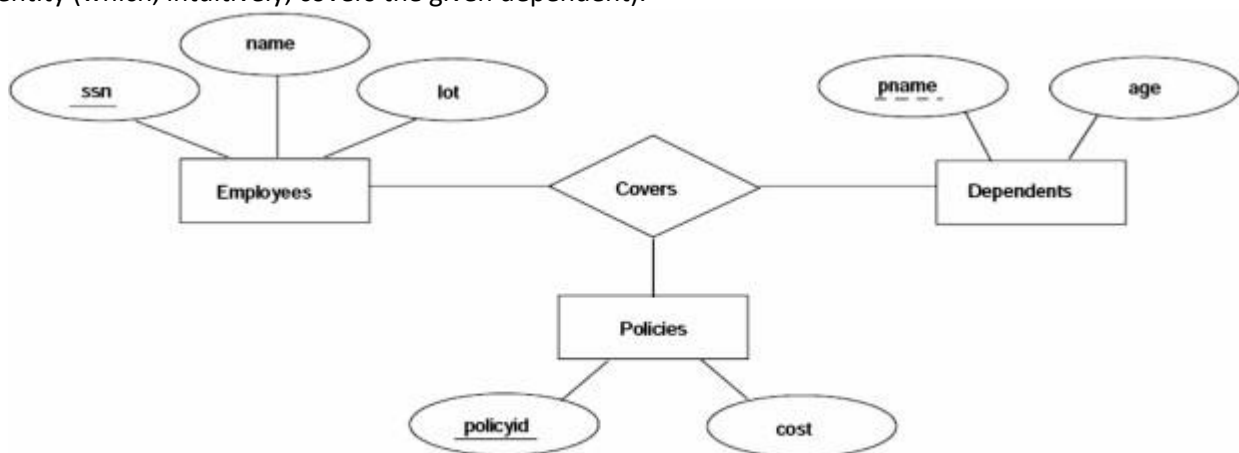


Figure 2.18 Policies as an Entity Set

The first requirement suggests that we impose a key constraint on Policies with respect to Covers, but this constraint has the unintended side effect that a policy can cover only one dependent. The second requirement suggests that we impose a total participation constraint on Policies. This solution is acceptable if each policy covers at least one dependent. The third requirement forces us to introduce an identifying relationship that is binary (in our version of ER diagrams, although there are versions in which this is not the case). Even ignoring the third point above, the best way to model this situation is to use two binary relationships, as shown in Figure 2.19.

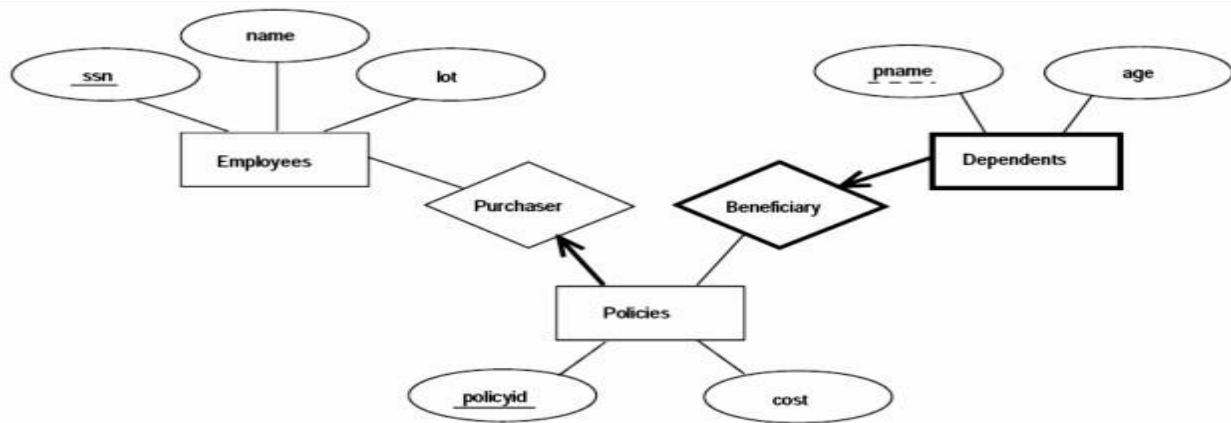


Figure 2.19 Policy Revisited

Aggregation versus Ternary Relationships *

- As we noted in Section 2.4.5, the choice between using aggregation or a ternary relationship is mainly determined by the existence of a relationship that relates a relationship set to an entity set (or second relationship set). The choice may also be guided by certain integrity constraints that we want to express. For example, consider the ERdiagram shown in Figure 2.13.

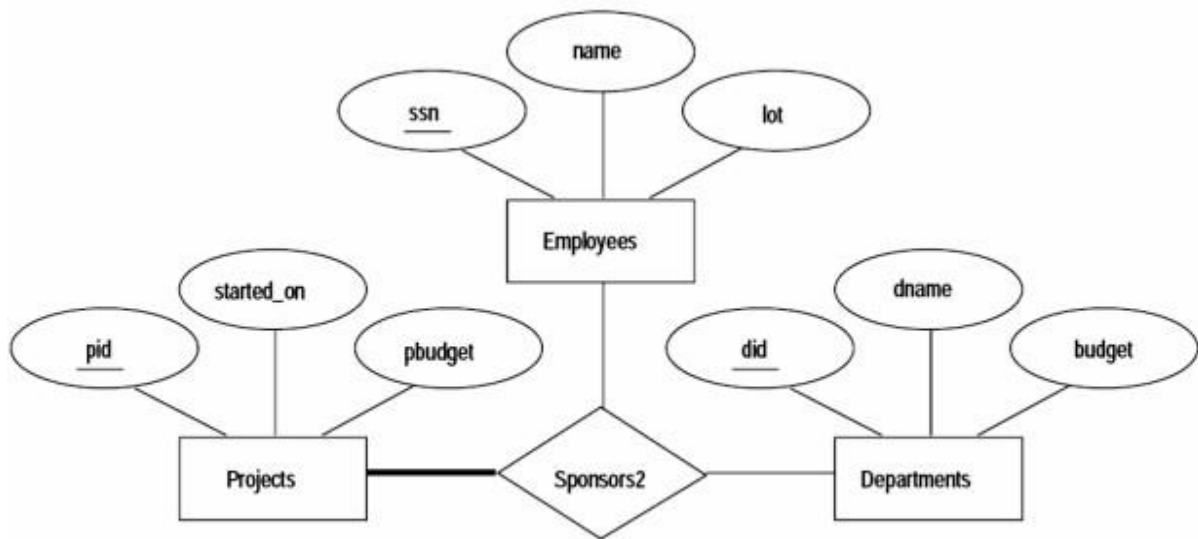


Figure 2.20 Using a Ternary Relationship instead of Aggregation

- According to this diagram, a project can be sponsored by any number of departments, a department can sponsor one or more projects, and each sponsorship is monitored by one or more employees. If we don't need to record the until attribute of Monitors, then we might reasonably use a ternary relationship, say, Sponsors2.

CONCEPTUAL DESIGN FOR LARGE ENTERPRISES *

- We have thus far concentrated on the constructs available in the ER model for describing various application concepts and relationships. The process of conceptual design consists of more than just describing small fragments of the application in terms of ER diagrams. For a large enterprise, the design may require the efforts of more than one designer and span data and application code used by a number of user groups. Using a high-level, semantic data model such as ER diagrams for conceptual design in such an environment offers the additional advantage that the high-level design can be diagrammatically represented and is easily understood by the many people who must provide input to the design process.
- An important aspect of the design process is the methodology used to structure the development of the overall design and to ensure that the design takes into account all user requirements and is consistent. The usual approach is that the requirements of various user groups are considered, any conflicting requirements are somehow resolved, and a single set of global requirements is generated at the end of the requirements analysis phase. Generating a single set of global requirements is a difficult task, but it allows the conceptual design phase

to proceed with the development of a logical schema that spans all the data and applications throughout the enterprise.

- An alternative approach is to develop separate conceptual schemas for different user groups and to then integrate these conceptual schemas. To integrate multiple conceptual schemas, we must establish correspondences between entities, relationships, and attributes, and we must resolve numerous kinds of conflicts (e.g., naming conflicts, domain mismatches, differences in measurement units). This task is difficult in its own right. In some situations schema integration cannot be avoided. For example, when one organization merges with another, existing databases may have to be integrated. Schema integration is also increasing in importance as users demand access to heterogeneous data sources, often maintained by different organizations

Advantages and Disadvantages of ER Modeling (Merits and Demerits of ER Modeling)

Advantages

- ER Modeling is simple and easily understandable. It is represented in business users language and it can be understood by non-technical specialist.
- Intuitive and helps in Physical Database creation.
- Can be generalized and specialized based on needs.
- Can help in database design.
- Gives a higher level description of the system.

Disadvantages

- Physical design derived from E-R Model may have some amount of ambiguities or inconsistency.
- Sometime diagrams may lead to misinterpretations